

Certyfikowany Tester

Poziom Zaawansowany

Tester Zwinny

(CTAL-AT)

Sylabus

wersja 2.0

wersja tłumaczenia PL 2.0.0



International Software Testing Qualifications Board



Informacja o prawach autorskich

© International Software Testing Qualifications Board (zwana dalej „ISTQB®”).

ISTQB® jest zarejestrowanym znakiem towarowym International Software Testing Qualifications Board.

Copyright © 2026 Autorzy wersji 2.0: Wim Decoutere, Michaël Pilaeten (Product Owner), Adam Roman, Murian Song, Nele Van Asch.

Prawa autorskie wersji polskiej: Copyright © Polish Quality Board (PQB).

Tłumaczenie z języka angielskiego: Adam Roman.

Przegląd tłumaczenia: Joanna Kazun, Adam Ścierański.

Przegląd tłumaczenia, korekta: Monika Petri-Starego.

Copyright © 2014 Autorzy wersji 1.0: Rex Black (przewodniczący), Anders Claesson, Gerry Coleman, Bertrand Cornanguer (wiceprzewodniczący), Istvan Forgacs, Alon Linetzki, Tilo Linz, Leo van der Aalst, Marie Walsh i Stephan Weber.

Wszelkie prawa zastrzeżone. Autorzy niniejszym przenoszą prawa autorskie na ISTQB®. Autorzy (jako obecni właściciele praw autorskich) i ISTQB® (jako przyszły właściciel praw autorskich) uzgodnili następujące warunki użytkowania:

- Fragmenty niniejszego dokumentu mogą być kopiowane do użytku niekomercyjnego, pod warunkiem podania źródła. Każdy akredytowany dostawca szkoleń może wykorzystać niniejszy sylabus jako podstawę szkolenia, pod warunkiem podania autorów i ISTQB® jako źródła i właścicieli praw autorskich do sylabusa oraz pod warunkiem, że wszelkie reklamy takiego szkolenia mogą zawierać wzmiankę o sylabusie dopiero po otrzymaniu oficjalnej akredytacji materiałów szkoleniowych od uznanej przez ISTQB® rady krajowej.
- Każda osoba lub grupa osób może wykorzystać niniejszy sylabus jako podstawę artykułów i książek, pod warunkiem podania autorów i ISTQB® jako źródła i właścicieli praw autorskich do sylabusa.
- Wszelkie inne wykorzystanie niniejszego sylabusa jest zabronione bez uprzedniej pisemnej zgody ISTQB®.
- Każda rada krajowa uznana przez ISTQB® może przetłumaczyć niniejszy sylabus pod warunkiem, że w przetłumaczonej wersji sylabusa zamieści powyższą informację o prawach autorskich.

Historia zmian

Wersja	Data	Uwagi
2.0 (2026)	17.04.2026	Generalna aktualizacja sylabusa
1.0 (2014)	31.05.2014	Pierwsza wersja (zastąpiona)

Historia zmian polskiej wersji dokumentu

Wersja	Data	Uwagi
2.0.0	6.10.2026	Opublikowanie polskiego tłumaczenia sylabusa

Spis treści

Informacja o prawach autorskich	2
Historia zmian	3
Historia zmian polskiej wersji dokumentu	3
Podziękowania	7
Wstęp	8
Cel niniejszego sylabusu	8
Tester zwinny w testowaniu oprogramowania	8
Ścieżki kariery dla testerów	8
Cele biznesowe	8
Cele nauczania objęte egzaminem oraz poziomy wiedzy	9
Egzamin certyfikacyjny Certyfikowany Tester Poziom Zaawansowany Tester Zwinny	9
Akredytacja	10
Odniesienia do norm	10
Poziom szczegółowości	11
Struktura sylabusu	11
1. Wyzwania związane ze strategią testów i podejściem do testów (60 min.)	12
Wprowadzenie do Agile	13
1.1. Typy testów	16
1.2. Testowanie kompleksowe	16
1.3. Testowanie formalne i testowanie holistyczne	17
1.4. Podejścia do testowania regresji	19
2. Ludzie i zespoły (60 min.)	21
Wstęp	22
2.1. Podejście „cały zespół”	22
2.1.1. Generalizacja i specjalizacja w ramach zespołu	22
2.1.2. Motywowanie interesariuszy biznesowych do udziału w testach	23
2.1.3. Wsparcie programistów	24
2.2. Testerzy jednorazowi	25
3. Zarządzanie testami i doskonalenie procesu testowego (210 min.)	26
Wstęp	27
3.1. Planowanie testów	28
3.1.1. Planowanie testów w zwinnym wytwarzaniu oprogramowania	28
3.1.2. Projektowa strategia testowa	29
3.2. Monitorowanie testów i nadzór nad testami	30

3.3.	Raportowanie testów	31
3.4.	Doskonalenie procesu testowego	32
3.4.1.	Metryki w doskonaleniu procesu testowego.....	32
3.4.2.	Doskonalenie procesu testowego w zwinnym wytwarzaniu oprogramowania	33
4.	Przesunięcie w lewo (135 min.)	35
	Wstęp.....	36
4.1.	Użycie przesunięcia w lewo w celu zwiększenia jakości podstawy testów	36
4.1.1.	Testalia jako wymagania	36
4.1.2.	Storyboarding i testboarding	37
4.1.3.	Mapowanie przykładów	37
4.1.4.	Stronniczość w testowaniu zwinnym	38
4.1.5.	Podział historyjki użytkownika	39
4.2.	Przesunięcie w lewo i inżynieria wymagań	40
5.	Zwinne podejścia do testów i techniki testowania (285 min.)	42
	Wstęp.....	43
5.1.	Testowanie eksploracyjne.....	43
5.1.1.	Heurystyki testowe	43
5.1.2.	Mnemoniki testowe	44
5.1.3.	Wędrowki testowe	44
5.1.4.	Tworzenie karty opisu testu.....	45
5.1.5.	Wykonywanie testowania eksploracyjnego	47
5.2.	Testowanie wspomagane	48
5.2.1.	Testowanie grupowe	48
5.2.2.	Testowanie w parach	49
5.2.3.	Testowanie intuicyjne	49
5.3.	Zapachy testów.....	50
6.	Automatyzacja testów i narzędzia testowe (30 min.).....	54
	Wstęp.....	55
6.1.	Automatyzacja testów w zwinnym wytwarzaniu oprogramowania	55
6.2.	Narzędzia testowe w zwinnym wytwarzaniu oprogramowania	56
7.	Bibliografia.....	58
8.	Pozostała literatura (niecytowana).....	60
9.	Dodatek A – lista skrótów.....	61
10.	Dodatek B – słowa kluczowe specyficzne dla dziedziny.....	62
11.	Dodatek C – cele nauczania i poziomy wiedzy	64
12.	Dodatek D – macierz powiązań między celami biznesowymi a celami nauczania.....	66
13.	Dodatek E – nota wydania	69

14.	Dodatek F – znaki towarowe	70
15.	Indeks	71

Podziękowania

Zgromadzenie Ogólne ISTQB® formalnie opublikowało ten dokument w dniu 17.04.2026 r. Został on opracowany przez zespół International Software Testing Qualifications Board w składzie: Wim Decoutere, Michaël Pilaeten, Adam Roman, Murian Song, Nele Van Asch.

Zespół dziękuje dr Stuartowi Reidowi i Markowi Rutzowi za przegląd techniczny, a także zespołowi recenzentów i radom krajowym za ich sugestie i uwagi.

W recenzowaniu, komentowaniu i głosowaniu nad niniejszym sylabusem uczestniczyły następujące osoby:

Abdellatif, Hatim Abuazab, Bíró Ádám, Tom Adams, Gergely Agnecz, Laura Albert, Tobias Anderson, Chris Van Bael, André Baumann, Jürgen Beniermann, Menno van den Berg, Ruud van Berkum, Armin Born, Earl Burba, Simeone Chiumarulo, Marco Ciarlito, Alessandro Collino, Walter Eder, Michael Fischlein, Valeriya Gagarina, Attila Gyuri, Ferenc Hamori, Andrea Horvath, Sagar Joshi, Mattijs Kemmink, Jędrzej Kwapiński, Thomas Letzkus, Ine Lutterman, Niranjan Maharajh, Judy McKay, Imre Meszaros, Gary Mogyorodi, Thaer Mustafa, Frank Neiryndck, Andreas Neumeister, Melanie Neumueller, Ingvar Nordström, Tauhida Parveen, Tal Pe'er, Christoph Peters, Lukas Piska, Horst Pohlmann, Andrew Pollner, Nishan Portoyan, Meile Posthuma, Randall Rice, Jean François Riverin, Nicola De Rosa, Mark Rutz, Sarno Salvatore, Márton Siska, Klaus Skafté, Péter Sótér, Michael Stahl, Lucjan Stapp, Ioannis Symeonidis, Reifa Tangon, Richard Taylor, Benjamin Timmermans, Giancarlo Tomasig, Stephanie Ulrich, Csilla Varga, Linda Vreeswijk i Jana Zientková.

Grupa Robocza ISTQB® ds. Poziomu Podstawowego w momencie publikacji sylabusa – rozszerzenia poziomu podstawowego o metodyki zwinne w wersji 1.0 (2014): Rex Black (przewodniczący), Bertrand Cornanguer (wiceprzewodniczący), Gerry Coleman (kierownik ds. celów nauczania), Debra Friedenberg (kierownik ds. egzaminów), Alon Linetzki (kierownik ds. wyników biznesowych i marketingu), Tauhida Parveen (redaktor) oraz Leo van der Aalst (kierownik ds. rozwoju).

Autorzy sylabusa – rozszerzenia poziomu podstawowego o metodyki zwinne w wersji 1.0 (2014): Rex Black, Anders Claesson, Gerry Coleman, Bertrand Cornanguer, Istvan Forgacs, Alon Linetzki, Tilo Linz, Leo van der Aalst, Marie Walsh oraz Stephan Weber.

Wewnętrzni recenzenci sylabusa – rozszerzenia poziomu podstawowego o metodyki zwinne w wersji 1.0 (2014): Mette Bruhn-Pedersen, Christopher Clements, Alessandro Collino, Debra Friedenberg, Kari Kakkonen, Beata Karpinska, Sammy Kolluru, Jennifer Leger, Thomas Mueller, Tuula Pääkkönen, Meile Posthuma, Gabor Puhalla, Lloyd Roden, Marko Rytönen, Monika Stoecklein-Olsen, Robert Treffny, Chris Van Bael oraz Erik van Veenendaal.

Zespół dziękuje również następującym osobom z rad krajowych oraz społeczności ekspertów Agile, które uczestniczyły w przeglądaniu, komentowaniu i głosowaniu nad sylabusem – rozszerzeniem poziomu podstawowego o metodyki zwinne w wersji 1.0 (2014): Dani Almog, Richard Berns, Stephen Bird, Monika Bögge, Afeng Chai, Josephine Crawford, Tibor Csöndes, Huba Demeter, Arnaud Foucal, Cyril Fumery, Kobi Halperin, Inga Hansen, Hanne Hinz, Jidong Hu, Phill Isles, Shirley Itah, Martin Klöck, Kjell Lauren, Igal Levi, Rik Marselis, Johan Meivert, Armin Metzger, Peter Morgan, Ninna Morin, Ingvar Nordstrom, Chris O'Dea, Klaus Olsen, Ismo Paukamainen, Nathalie Phung, Helmut Pichler, Salvatore Reale, Stuart Reid, Hans Rombouts, Petri Säilynoja, Soile Sainio, Lars-Erik Sandberg, Dakar Shalom, Jian Shen, Marco Sogliani, Lucjan Stapp, Yaron Tsubery, Sabine Uhde, Stephanie Ulrich, Tommi Välimäki, Jurian Van de Laar, Marnix Van den Ent, António Vieira Melo, Wenye Xu, Ester Zabar, Wenqiang Zheng, Peter Zimmerer, Stevan Zivanovic i Terry Zuo.

Wstęp

Cel niniejszego sylabusu

Niniejszy sylabus stanowi podstawę dla międzynarodowej kwalifikacji ISTQB® w zakresie testowania oprogramowania dla poziomu Certyfikowany Tester Poziom Zaawansowany Tester Zwinny. ISTQB® udostępnia niniejszy sylabus:

1. Radom krajowym w celu przetłumaczenia go na języki lokalne i akredytacji dostawców szkoleń. Rady krajowe mogą dostosować sylabus do swoich potrzeb językowych i zmodyfikować źródła bibliograficzne, aby dostosować je do lokalnych publikacji.
2. Organom certyfikującym w celu opracowania pytań egzaminacyjnych w ich lokalnym języku, dostosowanych do celów nauczania określonych w niniejszym sylabusie.
3. Organizatorom szkoleń w celu opracowania materiałów szkoleniowych i określenia odpowiednich metod nauczania.
4. Kandydatom do certyfikacji w celu przygotowania się do egzaminu certyfikacyjnego (w ramach szkoleń lub samodzielnie).
5. Międzynarodowej społeczności inżynierów oprogramowania i systemów w celu rozwoju zawodu testera oprogramowania i systemów oraz jako podstawa do tworzenia książek i artykułów.

Tester zwinny w testowaniu oprogramowania

Certyfikat ISTQB® Certyfikowany Tester Poziom Zaawansowany Tester Zwinny (CTAL-AT) jest przeznaczony dla wszystkich osób zaangażowanych w testowanie oprogramowania w projektach zwinnych. Obejmuje to osoby pełniące takie role jak: testerzy, analitycy testów, inżynierowie testów, konsultanci testów, kierownicy testów, testerzy akceptacji użytkownika oraz programiści.

Kwalifikacja ta jest również odpowiednia dla wszystkich osób pragnących lepiej zrozumieć testowanie oprogramowania w metodykach zwinnych, w tym właścicieli produktów, Scrum Masterów, kierowników ds. jakości, kierowników ds. rozwoju oprogramowania, analityków biznesowych, dyrektorów IT oraz konsultantów ds. zarządzania.

Ścieżki kariery dla testerów

Program ISTQB® zapewnia wsparcie dla specjalistów ds. testowania na wszystkich etapach ich kariery, oferując zarówno szeroką, jak i dogłębną wiedzę. Osoby, które uzyskają certyfikat ISTQB® Poziom Zaawansowany Tester Zwinny mogą być również zainteresowane innymi specjalistycznymi programami nauczania, ze szczególnym uwzględnieniem certyfikatu *Agile Test Leadership at Scale*.

Najnowsze informacje na temat programu certyfikacji testerów ISTQB® można znaleźć na stronie www.istqb.org.

Cele biznesowe

W tej sekcji wymieniono cele biznesowe (ang. *Business Outcomes*) osiąmane przez kandydata, który uzyskał certyfikat ISTQB® Certyfikowany Tester Poziom Zaawansowany Tester Zwinny.

Certyfikowany tester zwinny potrafi:

Kod	Opis celu biznesowego
CTAL-AT-BO1	Współpracować w zespołach wielofunkcyjnych, znając zasady i podstawowe praktyki zwinnego wytwarzania oprogramowania
CTAL-AT-BO2	Dostosować dotychczasowe doświadczenie i wiedzę z zakresu testowania do wartości i zasad Agile
CTAL-AT-BO3	Wspierać zespół zwinny w planowaniu testów
CTAL-AT-BO4	Stosować odpowiednie podejścia do wytwarzania oprogramowania w metodologii Agile oraz techniki testowania w celu zapewnienia testów o odpowiednim pokryciu
CTAL-AT-BO5	Pomóc interesariuszom biznesowym w definiowaniu zrozumiałych i możliwych do przetestowania historyjek użytkownika, scenariuszy, wymagań i kryteriów akceptacji, w zależności od potrzeb
CTAL-AT-BO6	Tworzyć i wdrażać różne podejścia do testowania w zwinnym wytwarzaniu oprogramowania
CTAL-AT-BO7	Wspierać i przyczyniać się do automatyzacji testów w projekcie zwinnego wytwarzania oprogramowania
CTAL-AT-BO8	Współpracować z innymi członkami zespołu oraz dzielić się z nimi informacjami przy użyciu skutecznych stylów i kanałów komunikacji

Cele nauczania objęte egzaminem oraz poziomy wiedzy

Cele nauczania (ang. *Learning Objectives*) wspierają cele biznesowe i są podstawą do tworzenia egzaminów certyfikacyjnych ISTQB[®] Certyfikowany Tester Poziom Zaawansowany Tester Zwinny.

Co do zasady, cała zawartość niniejszego programu nauczania podlega egzaminowi, z wyjątkiem wprowadzenia i załączników. Pytania egzaminacyjne weryfikują znajomość słów kluczowych na poziomie K2 (patrz poniżej) oraz cele nauczania na odpowiednim poziomie wiedzy.

Cele nauczania i ich poziomy wiedzy są przedstawione na początku każdego rozdziału i sklasyfikowano je w następujący sposób:

- K1 – zapamiętać (cele nauczania K1 nie występują w niniejszym sylabusie),
- K2 – zrozumieć,
- K3 – zastosować,
- K4 – przeanalizować.

Dalsze szczegóły i przykłady celów nauczania podano w rozdziale 11.

W przypadku wszystkich terminów wymienionych jako słowa kluczowe tuż pod nagłówkami rozdziałów należy zapamiętać (K1) ich prawidłową nazwę i definicję ze słownika ISTQB[®], a także zrozumieć ją, nawet jeśli konieczność jej znajomości nie została wyraźnie wymieniona w celu nauczania.

Egzamin certyfikacyjny Certyfikowany Tester Poziom Zaawansowany Tester Zwinny

Egzamin na certyfikat ISTQB[®] Certyfikowany Tester Poziom Zaawansowany Tester Zwinny jest oparty na niniejszym sylabusie. Odpowiedzi na pytania egzaminacyjne mogą wymagać wykorzystania materiałów opartych na więcej niż jednej sekcji niniejszego sylabusa. Wszystkie sekcje sylabusa podlegają egzaminowaniu, z wyjątkiem wprowadzenia i dodatków. Normy

i książki są uwzględnione jako odniesienia, ale ich treść nie podlega egzaminowaniu, poza tym, co zostało opisane w samym sylabusie.

Więcej szczegółów można znaleźć w dokumentach *Exam Structure and Rules* dostępnym na podstronie www.istqb.org dotyczącej sylabusa Certyfikowany Tester Poziom Zaawansowany Tester Zwinny.

Do egzaminu ISTQB® Certyfikowany Tester Poziom Zaawansowany Tester Zwinny mogą przystąpić:

- kandydaci zainteresowani testowaniem oprogramowania, zwłaszcza w środowiskach zwinnych,
- kandydaci posiadający certyfikat ISTQB® Poziom Podstawowy.

Zaleca się jednak, aby kandydaci również:

- posiadali przynajmniej minimalne doświadczenie w zakresie zwinnego wytwarzania oprogramowania lub testowania oprogramowania, np. sześciomiesięczne doświadczenie jako tester systemowy lub tester akceptacyjny użytkownika albo jako członek zespołu zwinnego,
- wzięli udział w szkoleniu akredytowanym przez jedną z uznanych rad krajowych ISTQB®, zgodnie ze standardami ISTQB®.

Akredytacja

Rada krajowa ISTQB® może akredytować dostawców szkoleń, których materiały szkoleniowe są zgodne z niniejszym sylabusem. Dostawcy szkoleń powinni uzyskać wytyczne dotyczące akredytacji od rady krajowej lub organu, który przeprowadza akredytację. Akredytowane szkolenie jest uznawane za zgodne z niniejszym sylabusem i umożliwia włączenie egzaminu ISTQB® do szkolenia. Wytyczne akredytacyjne dla niniejszego sylabusa są zgodne z ogólnymi wytycznymi akredytacyjnymi opublikowanymi przez ISTQB® *Processes Management and Compliance Working Group*.

Odniesienia do norm

Międzynarodowe organizacje normalizacyjne, takie jak IEEE i ISO, wydają normy związane z charakterystykami jakościowymi i testowaniem oprogramowania. Niektóre z tych norm są przywoływane w niniejszym sylabusie. Celem tych odniesień jest zapewnienie ram pojęciowych (jak w odniesieniach do normy ISO/IEC 25010 dotyczącej charakterystyk jakościowych oprogramowania) lub źródła dodatkowych informacji, jeśli jest to pożądane przez czytelnika. Należy pamiętać, że sylabusy ISTQB® wykorzystują normy jako odniesienie. Zawartość tych dokumentów nie podlega egzaminowaniu. Więcej informacji o normach znajduje się w Rozdziale 6.

Poziom szczegółowości

Poziom szczegółowości niniejszego sylabusa umożliwia prowadzenie szkoleń i egzaminów spójnych w skali międzynarodowej. Aby osiągnąć ten cel, w sylabusie uwzględniono:

- ogólne cele dydaktyczne opisujące założenia programu nauczania na poziomie Certyfikowany Tester Poziom Zaawansowany Tester Zwinny,
- listy słów kluczowych, których definicje kandydaci muszą pamiętać,
- cele nauczania dla każdego obszaru wiedzy, opisujące poznawcze efekty uczenia się, które mają zostać osiągnięte przez kandydatów,
- opis kluczowych pojęć, w tym odniesień do źródeł, takich jak uznana literatura lub normy.

Zawartość sylabusa nie opisuje całego obszaru wiedzy na temat testowania oprogramowania; odzwierciedla raczej poziom szczegółowości, który ma zostać omówiony na szkoleniach dla testerów zwinnych na poziomie zaawansowanym. Koncentruje się ona na podejściach i technikach testowania, które można zastosować we wszystkich projektach oprogramowania realizowanych zgodnie z metodami zwinnymi.

Terminologia (tj. terminy i ich znaczenie) z zakresu testowania i zapewniania jakości oprogramowania wykorzystywana w niniejszym sylabusie jest zgodna ze Słownikiem terminów testowych ISTQB® (ISTQB Glossary).

Terminologię dotyczącą dyscyplin powiązanych można znaleźć w Słowniku inżynierii wymagań IREB (IREB-CPRE) oraz Słowniku inżynierii oprogramowania IEEE (IEEE-Pascal) oraz *Agile Alliance* dla terminologii Agile (*Agile Alliance*).

Struktura sylabusa

Sylabus zawiera sześć rozdziałów z treściami podlegającymi egzaminowaniu. W nagłówku każdego rozdziału określony jest czas przeznaczony na omówienie tego rozdziału podczas szkolenia. Czas ten nie jest podawany poniżej poziomu rozdziału. W przypadku akredytowanych szkoleń sylabus wymaga co najmniej 13 godzin zajęć (690 minut), rozłożonych na sześć rozdziałów w następujący sposób:

- Rozdział 1: Wyzwania związane ze strategią testów i podejściem do testów – 60 minut.
- Rozdział 2: Ludzie i zespoły – 60 minut.
- Rozdział 3: Zarządzanie testami i doskonalenie procesu testowego – 210 minut.
- Rozdział 4: Przesunięcie w lewo – 135 minut.
- Rozdział 5: Zwinne podejścia do testów i techniki testowania – 285 minut.
- Rozdział 6: Automatyzacja testów i narzędzia – 30 minut.

1. Wyzwania związane ze strategią testów i podejściem do testów (60 min.)

Słowa kluczowe

atak na defekty (ang. *bug bash*), przypadek skrajny (ang. *edge case*), testowanie formalne, testowanie holistyczne, testowanie kompleksowe (ang. *end-to-end testing*), testowanie regresji, typ testu, wytwarzanie sterowane testami, wytwarzanie sterowane testami akceptacyjnymi

Słowa kluczowe specyficzne dla dziedziny

iteracja, iteracja wzmacniająca (ang. *hardening iteration*), ścieżka użytkownika (ang. *user journey*), przełącznik funkcji (ang. *feature toggle*)

Cele nauczania dla rozdziału 1

1.1 Typy testów

AT-1.1.1 (K2) Porównać typy testów wykonywane w trakcie i po iteracji.

1.2 Testowanie kompleksowe

AT-1.2.1 (K2) Wyjaśnić, kiedy powinno być wykonywane testowanie kompleksowe.

1.3 Testowanie formalne i testowanie holistyczne

AT-1.3.1 (K2) Porównać korzyści i wady testowania formalnego i testowania holistycznego.

1.4 Podejścia do testowania regresji

AT-1.4.1 (K2) Rozróżniać podejścia do testowania regresji.

Wprowadzenie do Agile

Podstawy metodyki Agile są opisane w sylabusie poziomu podstawowego, jednak niniejsze obszernie wprowadzenie uzupełnia go o istotne pojęcia, które nie zostały tam uwzględnione, a które są kluczowe dla zrozumienia praktyk, ról i sposobu myślenia omawianych w niniejszym sylabusie.

Zwinne wytwarzanie oprogramowania jest obecnie jednym z najczęściej stosowanych podejść w branży IT. Jego głównymi celami są szybkie dostosowywanie się do zmian oraz konsekwentne dostarczanie wartości klientom. Aby osiągnąć te cele, zwinne wytwarzanie oprogramowania przyjmuje podejście odmienne od sekwencyjnych modeli wytwarzania oprogramowania, co doprowadziło do powstania testowania zwinnego (ang. *Agile testing*). Zwinne testowanie wykracza poza zwykłe testowanie oprogramowania; jest to oparta na współpracy, zintegrowana metodyka, która zapewnia jakość w całym procesie rozwoju i podkreśla odpowiedzialność całego zespołu za jakość.

Manifest zwinnego wytwarzania oprogramowania. W 2001 roku grupa osób reprezentujących najczęściej stosowane lekkie metodyki wytwarzania oprogramowania uzgodniła wspólny zestaw wartości i zasad, który stał się znany jako Manifest zwinnego wytwarzania oprogramowania lub Manifest Agile (Beck; Beedle i in., 2001). Manifest Agile definiuje następujące wartości:

- **Ludzie i interakcje ponad procesy i narzędzia.** Zwinne wytwarzanie oprogramowania jest zorientowane na ludzi. Oprogramowanie tworzą zespoły ludzi, które pracują najskuteczniej dzięki ciągłej komunikacji i interakcji, a nie poleganiu na narzędziach czy procesach.
- **Działające oprogramowanie ponad wyczerpującą dokumentację.** Z punktu widzenia klienta działające oprogramowanie jest znacznie bardziej użyteczne i wartościowe niż nadmiernie szczegółowa dokumentacja. Daje także klientowi możliwość szybkiego przekazania zespołowi twórczemu informacji zwrotnej. Ponadto, ponieważ działające oprogramowanie, nawet o ograniczonej funkcjonalności, jest dostarczane wcześniej w cyklu życia wytwarzania oprogramowania, zwinne wytwarzanie oprogramowania może zapewnić znaczną przewagę w zakresie czasu wprowadzenia produktu na rynek. Zwinne wytwarzanie oprogramowania jest szczególnie przydatne w szybko zmieniających się środowiskach biznesowych, gdzie problemy lub rozwiązania są niejasne lub gdzie firma chce wprowadzać innowacje w nowych obszarach dziedzinowych.
- **Współpraca z klientem ponad negocjacje umów.** Klientom często trudno jest dokładnie określić wymagania dotyczące systemu. Bezpośrednia współpraca z klientem zwiększa prawdopodobieństwo dokładnego zrozumienia jego potrzeb. Chociaż umowy z klientami mogą być ważne, regularna, ścisła współpraca z nimi prawdopodobnie przyniesie większy sukces projektu.
- **Reagowanie na zmiany ponad ścisłe podążanie za planem.** Zmiany w projektach są nieuniknione. Środowisko, w którym działa firma, w tym przepisy prawne, działania konkurencji, postęp technologiczny i inne czynniki, mogą znacząco wpłynąć na projekt i jego cele. Czynniki te należy uwzględnić w procesie wytwarzania oprogramowania. W związku z tym elastyczność w praktykach pracy, pozwalająca na dostosowanie się do zmian, jest ważniejsza niż ścisłe przestrzeganie planu.

Zasady Agile. Dwanaście zasad Manifestu Agile oddaje jego podstawowe wartości, kładąc nacisk na wczesne i częste dostarczanie wartościowego oprogramowania, dostosowywanie się do zmieniających się wymagań, częste wydania oraz codzienną współpracę między interesariuszami biznesowymi a programistami. Zasady te priorytetowo traktują zmotywowane

i wspierane osoby, bezpośrednią komunikację, działające oprogramowanie jako kluczowy miernik postępu oraz zrównoważony rozwój w statym tempie. Podkreślają doskonałość techniczną, prostotę, samoorganizujące się zespoły oraz regularną refleksję w celu ciągłego zwiększania efektywności i zdolności adaptacyjnych.

Praktyki zwinnego wytwarzania oprogramowania. Zespoły zwinne stosują różne praktyki, aby wprowadzić te wartości i zasady w życie. Typowe praktyki stosowane w większości organizacji zwinnych obejmują wspólne tworzenie historyjek użytkownika, retrospektywy, ciągłą integrację, planowanie iteracji oraz planowanie wydania.

Metodyki zwinne. Organizacje stosują różne metodyki zwinnego wytwarzania oprogramowania. Najbardziej reprezentatywne z nich to:

- **Scrum** – lekka struktura, która organizuje pracę w iteracje o stałej długości zwane sprintami, kładąc nacisk na zdefiniowane role, ceremonie i spriorytetyzowany backlog produktu (ang. *product backlog*) (Sutherland i in., 2020).
- **Extreme Programming (XP)** – metodyka skupiona na praktykach inżynierskich, takich jak wytwarzanie sterowane testami, ciągła integracja i ścisła współpraca w celu poprawy jakości oprogramowania i jego responsywności (Beck; Andres, 2004).
- **Kanban** – wizualna metoda zarządzania przepływem pracy, która ogranicza liczbę zadań w toku (ang. *work in progress*), kładzie nacisk na ciągłe dostarczanie oraz wykorzystuje tablice do śledzenia i optymalizacji przepływu (Anderson i in., 2010).

Definicja testowania zwinnego. Janet Gregory i Lisa Crispin, czołowe postaci społeczności Agile, definiują testowanie zwinne jako „wspólne praktyki testowania, które mają miejsce od momentu powstania projektu aż do jego dostarczenia, wspierające częste dostarczanie wysokiej jakości produktów, które zwiększają wartość biznesową dla naszych klientów. Czynności testowe skupiają się na zapobieganiu defektom, a nie na ich wykrywaniu oraz mają na celu wzmocnienie i wsparcie idei odpowiedzialności całego zespołu za jakość” (Gregory; Crispin, 2019).

Definicja ta jasno określa podstawowe zasady testowania zwinnego. Nie ogranicza się ono do końcowych etapów projektu; testowanie jest raczej działaniem ciągłym, prowadzonym od samego początku. Ponadto rola testowania ma charakter wspólny i zmienia się ze zwykłego wykrywania defektów na proaktywne zapobieganie im. Odzwierciedla to podejście „cały zespół” (ang. *whole team approach*), w którym wszyscy zaangażowani w proces rozwoju dzielą odpowiedzialność za jakość.

Testowanie zwinne obejmuje szeroki zakres działań, w tym – ale nie tylko – kierowanie rozwojem za pomocą konkretnych przykładów, kwestionowanie pomysłów i założeń za pomocą wnikliwych pytań, automatyzację testów w celu poprawy wydajności, przeprowadzanie testów eksploracyjnych w celu wykrycia defektów oraz ocenę charakterystyk jakościowych takich jak wydajność, niezawodność i zabezpieczenia. Działania te pokazują, że w zwinnym wytwarzaniu oprogramowania rola testerów nie ogranicza się jedynie do wykonywania testów; są oni moderatorami (ang. *facilitators*) jakości i współpracownikami (ang. *collaborators*).

Manifest testowania zwinnego. Manifest testowania zwinnego autorstwa Karen Greaves i Samantha Laing zawarł wartości i zasady, które kierują testowaniem zwinnym, w pięciu zwięzłych stwierdzeniach.

Manifest ten podkreśla podstawowe różnice w stosunku do testowania w sekwencyjnym modelu wytwarzania oprogramowania i nakreśla sposób myślenia kluczowy dla ról testujących w środowisku zwinnym (Greaves i in., 2019):

- Testowanie przez cały czas zamiast testowania na końcu. W sekwencyjnym wytwarzaniu oprogramowania testowanie często rozpoczyna się dopiero po zakończeniu rozwoju. Agile

kładzie jednak nacisk na ciągłe testowanie w całym cyklu życia oprogramowania, od analizy wymagań i projektowania po wdrożenie i uruchomienie. Takie podejście pomaga wcześniej identyfikować i usuwać defekty, co znacznie zmniejsza koszty i nakład pracy. Dzięki przyjęciu podejścia przesunięcie w lewo cały zespół buduje jakość od samego początku, czyniąc testowanie ciągłym procesem uczenia się, który wzmacnia zaufanie do testowanego oprogramowania.

- Zapobieganie defektom zamiast ich wykrywania. Ostatecznym celem testowania w podejściu Agile nie jest tylko wykrywanie defektów, ale przede wszystkim zapobieganie ich powstawaniu. Aby to osiągnąć, osoby odpowiedzialne za testowanie aktywnie uczestniczą we wczesnych dyskusjach, takich jak wyjaśnianie wymagań i przegląd projektów, dzięki czemu proaktywnie identyfikują i ograniczają potencjalne problemy.
- Testowanie zrozumienia zamiast sprawdzania funkcjonalności. Oprócz zwykłego sprawdzania, czy oprogramowanie działa zgodnie z deklarowanymi funkcjami, kluczowe jest testowanie podstawowego zrozumienia celu oprogramowania i potrzeb użytkowników. Obejmuje to ocenę systemu z perspektywy użytkownika, często poprzez historyjki użytkownika i scenariusze, aby upewnić się, że jest on zgodny z wartością biznesową.
- Budowanie najlepszego systemu zamiast niszczenia go. Głównym zadaniem testerów nie jest wykorzystywanie słabości systemu ani celowe jego niszczenie. Zamiast tego współpracują oni z programistami, aby pomóc w stworzeniu systemu najwyższej jakości. Obejmuje to przekazywanie konstruktywnych opinii, proponowanie testowalnych projektów oraz dzielenie się pomysłami na ciągłe doskonalenie jakości.
- Odpowiedzialność zespołu za jakość zamiast odpowiedzialności testerów. Jakość nie jest wyłączną odpowiedzialnością pojedynczej osoby lub dedykowanego zespołu testowego. Jest to wspólne zobowiązanie wszystkich osób zaangażowanych w proces rozwoju, w tym programistów, przedstawicieli jednostek biznesowych, testerów i innych interesariuszy. Testowanie zwinne sprzyja tej kulturze wspólnej odpowiedzialności, zachęcając każdego członka zespołu do aktywnego udziału w działaniach związanych z jakością.

Znaczenie i wpływ testowania zwinnego. Te podstawowe zasady testowania zwinnego mają kluczowe znaczenie dla zapewnienia jakości oprogramowania w ramach zwinnego wytwarzania oprogramowania, które stawia na pierwszym miejscu szybkość reakcji i natychmiastową informację zwrotną. W sekwencyjnym wytwarzaniu oprogramowania testowanie często staje się wąskim gardłem projektu lub ostatnią „bramką jakości” tuż przed wydaniem. W podejściu zwinnym testowanie jest zintegrowane z wytwarzaniem oprogramowania jako fundamentalna i nieodzowna część tego procesu. Ta integracja pozwala zespołom konsekwentnie dostarczać oprogramowanie wysokiej jakości.

Testowanie zwinne znacznie wzmacnia również współpracę i komunikację w zespole. Osoby odpowiedzialne za testowanie ściśle współpracują z programistami i przedstawicielami biznesu, aby wyjaśnić wymagania, wcześniej zidentyfikować potencjalne defekty i zachęcić wszystkich do wspólnego dążenia do jakości. To środowisko oparte na współpracy nie tylko przyczynia się do zapobiegania defektom, ale także odgrywa kluczową rolę w zwiększaniu produktywności zespołu i ogólnej wydajności. Rola testera w tym podejściu niekoniecznie jest powiązana z konkretnym stanowiskiem w organizacji – może ją pełnić każdy w dowolnym momencie.

W istocie testowanie zwinne to coś więcej niż tylko zbiór technik testowania; stanowi ono głęboką zmianę w kulturze wytwarzania oprogramowania i w sposobie myślenia. Skupia się na wartości dla klienta, elastycznym dostosowywaniu się do zmian oraz ciągłym dostarczaniu produktów wysokiej jakości w ramach wspólnej odpowiedzialności całego zespołu.

1.1. Typy testów

Typy testów wybiera się w oparciu o konkretne cele testów i kontekst iteracji, w tym kryteria akceptacji historyjek użytkownika, ryzyka jakościowe oraz potrzeby w zakresie informacji zwrotnej. Wszystkie typy testów mogą być stosowane na każdym etapie projektu, ale ich wykorzystanie może się różnić w trakcie iteracji i po jej zakończeniu. Każda podjęta decyzja powinna znaleźć odzwierciedlenie w definicji ukończenia (ang. *Definition of Done*) oraz w podejściu do testów.

Podczas iteracji testy funkcjonalne sprawdzają, czy zaimplementowane funkcje spełniają odpowiednie wymagania funkcjonalne i kryteria akceptacji. Po zakończeniu iteracji rozszerzone testy funkcjonalne mogą obejmować testy funkcjonalne integracji różnych funkcji, w tym tych opracowanych przez inne zespoły.

Podczas iteracji testy нефункционалне zazwyczaj ograniczają się do sprawdzania podstawowej wydajności, użyteczności lub zabezpieczeń nowych funkcji. Po zakończeniu iteracji można przeprowadzić pełne testy нефункционалне w środowiskach zintegrowanych lub przedprodukcyjnych (ang. *staging*). Po zakończeniu iteracji można również przeprowadzić dalsze testy eksploracyjne i testy użyteczności z udziałem szerszej grupy, aby zweryfikować przydatność produktu do zamierzonego celu.

Podczas iteracji testy białoskrzynkowe są stosowane przed lub w trakcie kodowania, na przykład w ramach wytwarzania sterowanego testami (TDD). Wspiera to wczesne wykrywanie defektów i pomaga utrzymać stabilną bazę kodu. Testy te rzadko przeprowadza się jako oddzielne działanie po zakończeniu iteracji – jednak podczas wykonywania testów czarnoskrzynkowych można mierzyć pokrycie kodu, aby uzyskać dodatkowy wgląd w ich skuteczność i kompletność.

Podczas iteracji testowanie czarnoskrzynkowe koncentruje się na weryfikacji nowych lub zmodyfikowanych funkcji w odniesieniu do specyfikacji zdefiniowanych zazwyczaj jako kryteria akceptacji w historyjkach użytkownika. Jest to zgodne z testowaniem funkcjonalnym z wykorzystaniem wytwarzania sterowanego testami akceptacyjnymi (ATDD), co pomaga zapewnić, że dostarczona funkcjonalność spełnia oczekiwania interesariuszy. Po zakończeniu iteracji można przeprowadzić szersze testy czarnoskrzynkowe na poziomie testów systemowych, w tym testy regresji i testy kompleksowe (ang. *end-to-end*).

Podczas iteracji kluczową rolę odgrywają testy eksploracyjne. To specyficzne podejście do testowania funkcjonalnego uzupełnia zarówno testy białoskrzynkowe, jak i czarnoskrzynkowe, ujawniając nieznane ryzyka i wyłaniające się zachowania poprzez ustrukturyzowane sesje o ograniczonym czasie trwania.

Informacje dotyczące powiązania między typami testów a kwadrantami testowymi znajdują się w sekcji 3.1.2.

1.2. Testowanie kompleksowe

W zwinnym wytwarzaniu oprogramowania testy kompleksowe (ang. *end-to-end testing*, E2E) przeprowadza się wtedy, gdy wnoszą one wartość dodaną poprzez weryfikację integracji między systemami i ścieżkami działania użytkowników. Obejmuje to testowanie całej aplikacji oraz, w razie potrzeby, wielu systemów lub usług. Jednak jako działanie łagodzące ryzyko należy je rozważyć w kontekście ryzyka związanego z łańcuchem E2E w odniesieniu do konkretnej historyjki użytkownika, ponieważ ich utrzymanie jest kosztowne, a przeprowadzenie czasochłonne; ponadto, testy kompleksowe zapewniają ograniczoną informację diagnostyczną. Testy kompleksowe są zazwyczaj przeprowadzane bliżej terminu wydania, podczas iteracji wzmacniających (ang. *hardening iteration*), gdy wprowadzane są nowe punkty integracji w całym

systemie lub gdy istnieje wysokie ryzyko awarii. Chociaż iteracje wzmacniające są stosowane w niektórych organizacjach, mogą one wskazywać na niewystarczającą dbałość o jakość podczas regularnych iteracji. Testy kompleksowe są szczególnie przydatne w systemach rozproszonych, złożonych ścieżkach użytkownika lub scenariuszach z zależnościami od stron trzecich, których nie można niezawodnie przetestować w izolacji. Ich celem jest walidacja krytycznych scenariuszy użytkownika w środowisku przedprodukcyjnym, a nie wykrywanie defektów niskiego poziomu.

Na decyzję o włączeniu testów kompleksowych wpływa kilka czynników:

- Ryzyko. Wpływ ryzyka i prawdopodobieństwo ryzyka wystąpienia awarii na poziomie systemu uzasadniają przeprowadzenie ukierunkowanych testów kompleksowych, nawet jeśli zaufanie budowane jest poprzez testy niższego poziomu.
- Czas uzyskania informacji zwrotnej. Duże poleganie na testach *end-to-end* spowalnia dostarczanie. Zespoły często preferują szybsze testy na niższych poziomach i wybierają tylko kilka kluczowych przepływów do pokrycia testami kompleksowymi.
- Dojrzałość procesu ciągłej integracji/ciągłego dostarczania (CI/CD). Gdy zespoły dysponują dobrą automatyzacją testów, obserwowalnością i monitorowaniem w środowisku produkcyjnym, zaufanie może przesunąć się w kierunku informacji zwrotnej z produkcji, przełączników funkcji (ang. *feature toggles*) lub testów A/B jako alternatyw lub uzupełnienia rozbudowanych testów kompleksowych.

Z punktu widzenia strategii testów piramida testów pozostaje aktualna: szeroka, szybka informacja zwrotna u podstawy (testowanie modułowe), mniej testów na poziomie usług i minimalna liczba testów kompleksowych na szczycie. Odstępstwo od tej struktury może zwiększyć koszty i ryzyko bez proporcjonalnej wartości.

W mikroustugach i architekturach rozproszonych testowanie kontraktowe (ang. *contract testing*) stanowi alternatywę dla rozbudowanych testów kompleksowych. Testowanie kontraktowe weryfikuje, czy usługi komunikują się poprawnie, poprzez weryfikację uzgodnionych interfejsów (kontraktów) między konsumentami a dostawcami usług. W przeciwieństwie do testów kompleksowych, które wymagają uruchomienia wszystkich usług, testy kontraktowe mogą być wykonywane niezależnie, zapewniając szybszą informację zwrotną i łatwiejszą diagnostykę awarii. Testy kontraktowe zorientowane na konsumenta, w których konsumenci definiują swoje oczekiwania, a dostawcy weryfikują, czy są one spełnione, są szczególnie skuteczne w środowiskach zwinnych, gdzie usługi ewoluują niezależnie. Testy kontraktowe uzupełniają, a nie zastępują, niewielki zestaw testów kompleksowych, które weryfikują krytyczne ścieżki użytkownika.

Testy integracyjne powinny być najlepiej przeprowadzane na poziomie interfejsu, chyba że wprost wpływa to na zachowania na poziomie systemu. Testy kompleksowe powinny być zgodne z celami jakościowymi zdefiniowanymi przez interesariuszy i nie powinny być stosowane bezkrytycznie (ISO 29119-6, 2021).

Oskryptowane testy kompleksowe mogą pomijać nieoczekiwane zachowania wynikające z rzeczywistego użytkownika. Testowanie holistyczne (patrz podrozdział 1.3) i testowanie eksploracyjne (patrz sekcja 5.1.5) uzupełniają te testy, umożliwiając testerom badanie jakości z wielu perspektyw oraz wykorzystanie testowania opartego na doświadczeniu w celu wykrycia nieoczekiwanych zachowań, defektów użyteczności i przypadków skrajnych, które mogą zostać pominięte przez predefiniowane skrypty.

1.3. Testowanie formalne i testowanie holistyczne

Testowanie formalne kładzie nacisk na precyzję i formalizację. Polega ono na wyprowadzaniu przypadków testowych z jasno zdefiniowanych wymagań. Zazwyczaj stosuje się je w kontekstach

wymagających ścisłej zgodności lub kontroli ryzyka (np. testy regresji, gdy regresja jest uważana za ryzyko).

Korzyści płynące z testowania formalnego obejmują:

- Śledzenie powiązań – zapewnione jest śledzenie powiązań pomiędzy wymaganiami a przypadkami testowymi.
- Dostosowanie – przypadki testowe są dostosowane do miar pokrycia.
- Weryfikacja – testowanie formalne pomaga w weryfikacji, sprawdzając, czy system został zbudowany prawidłowo.
- Zgodność – przypadki testowe można sprawdzić, aby zapewnić zgodność z normami i wytycznymi.
- Powtarzalność – ponieważ przypadki testowe są formalnie udokumentowane, zapewniona jest powtarzalność.
- Automatyzacja testów – formalizacja i precyzja ułatwiają przejście do automatyzacji testów.

Wyzwania związane z testowaniem formalnym obejmują:

- Nadmierną sztywność – testowanie formalne ma tendencję do ograniczania eksploracji i zdolności reagowania na zmiany.
- Nadmierne poleganie na udokumentowanych wymaganiach – podstawa testów jest zazwyczaj niekompletna, niespójna lub nieprawidłowa. Gdy przypadki testowe są tworzone wyłącznie na podstawie słabych lub wadliwych wymagań, skutkuje to niekompletnym pokryciem, niedopasowanymi przypadkami testowymi i zwiększonym prawdopodobieństwem niewykrytych defektów.
- Pominięcie nieoczekiwanych zdarzeń – skupiając się wyłącznie na oczekiwanych wynikach, można przeoczyć pojawiające się ryzyka i awarie.
- Opóźniona informacja zwrotna – testy są często przeprowadzane na późnym etapie cyklu testowego, co opóźnia wykrycie krytycznych problemów.
- Niskie zaangażowanie poznawcze – testowanie formalne zachęca do mechanicznego wykonywania testów zamiast do przemyślanej analizy.

Testowanie holistyczne (Gregory, 2021) wykracza poza zaplanowane działania testowe i obejmuje całościowy obraz jakości systemu. Obejmuje ono ciągłe testowanie w całym cyklu życia oprogramowania i uwzględnia szeroki zakres perspektyw: klienta, zespołu i organizacji. Koncentruje się na współpracy, świadomości kontekstu, myśleniu systemowym (patrz (ISTQB-ATLAS, w. 1.0), sekcja 3.2.1) oraz eksploracji.

Korzyści płynące z testowania holistycznego obejmują:

- Uwzględnienie różnych perspektyw – jednocześnie połączenie punktów widzenia opartych na procesie, produkcie i użytkowniku.
- „Dobrze za pierwszym razem” (ang. *Right First Time*, RFT) – zapewnienie, że jakość jest wbudowana, a nie tylko testowana po fakcie.
- Ciągłe testowanie – łączy weryfikację (budowanie systemu poprawnie) i walidację (budowanie poprawnego systemu).

Wyzwania związane z testowaniem holistycznym obejmują:

- Brak skupienia – bez zdyscyplinowanej struktury zespoły mogą przeprowadzać szeroko zakrojone testy, ale przeoczyć krytyczne ryzyka lub regresje.
- Trudność pomiaru – ocena jakości zależy od kontekstu i nie ma ustalonych metryk, co osłabia zaufanie interesariuszy.

- Wymaga zmiany kultury – opiera się na ścisłej współpracy i myśleniu systemowym, a bez wspierającego przywództwa i dojrzałości zespołu wdrożenie może zakończyć się niepowodzeniem.
- Niejasność odpowiedzialności – odpowiedzialność „całego zespołu” może osłabić poczucie odpowiedzialności, jeśli nie jest ona starannie zarządzana.
- Duża zależność od kontekstu – nie ma uniwersalnego zastosowania i może nie spełniać wymogów regulacyjnych lub umownych, które wymagają bardziej formalnej weryfikacji.
- Problemy z automatyzacją testów – brak formalności zagraża prawidłowej automatyzacji testów.

Aby zmaksymalizować korzyści płynące z tych dwóch podejść, można je stosować w sposób komplementarny. Testy formalne powinny koncentrować się na krytycznych modułach, obszarach wysokiego ryzyka lub wymogach regulacyjnych i prawnych oraz być w miarę możliwości zautomatyzowane, aby zminimalizować powtarzalne wysiłki. Testy holistyczne powinny być stosowane do ogólnej kontroli jakości, wczesnej informacji zwrotnej i badania złożonych scenariuszy, z priorytetowym traktowaniem obszarów o największym wpływie na użytkowników.

1.4. Podejścia do testowania regresji

W zwinnym wytwarzaniu oprogramowania testy regresji muszą być zorganizowane w sposób umożliwiający ciągłe dostarczanie oprogramowania i szybkie wprowadzanie zmian. W zależności od kontekstu, możliwości zespołu, profilu ryzyka oraz stopnia zaawansowania automatyzacji testów można stosować różne podejścia do testów regresji, zarówno osobno, jak i łącznie:

- **Testy regresji przyrostowe** opierają się na zautomatyzowanym zestawie testowym, który jest stale utrzymywany i uruchamiany. Zamiast ponownego uruchamiania pełnego zestawu testów regresji dopiero na końcu iteracji lub wydania, podzbiór testów regresji jest uruchamiany automatycznie przy każdym zatwierdzeniu kodu lub zdarzeniu integracyjnym. Podzbiór ten jest wybrany na podstawie wpływu ryzyka, prawdopodobieństwa ryzyka, lokalizacji i zależności w zmianach kodu. Pozwala to na wczesne wykrywanie defektów regresji i wspiera ciągłe pętle informacji zwrotnej.
- **Testy regresji oparte na ryzyku** wykorzystują częste warsztaty oceny ryzyka lub dyskusje zespołowe w celu zidentyfikowania obszarów, w których zmiany mogą spowodować defekty, i kierują zarówno zautomatyzowanym, jak i manualnym testowaniem regresji. Metoda ta pomaga zdecydować, gdzie automatyzacja testów jest zbędna, a gdzie badanie przeprowadzone przez człowieka wnosi większą wartość.
- **Testy regresji zorientowane na DevOps** wplatają testy regresji w potok CI/CD, gdzie testy pełnią rolę bramek jakości, a testy regresji o wysokim priorytecie są wykonywane automatycznie po wdrożeniu w środowiskach przedprodukcyjnych lub nawet produkcyjnych; przełączniki funkcji i wydania kanarkowe (ang. *canary releases*) pozwalają na kontrolowane udostępnianie zmian, a obserwowalność i monitorowanie mogą w niektórych przypadkach zastąpić tradycyjne testy regresji.
- **Eksploracyjne testy regresji** są stosowane w sposób ciągły lub w regularnych odstępach czasu, uzupełniając automatyzację testów poprzez wykorzystanie spostrzeżeń testerów do badania interakcji między funkcjami i wykrywania nieoczekiwanych defektów.
- **Testowanie regresji oparte na współpracy** obejmuje czynności testowe, takie jak atak na defekty (ang. *bug bashes*) lub przeglądy krytycznych przepływów przez cały zespół, zwiększając świadomość zespołu na temat aktualnego stanu produktu i obszarów

ryzyka. Może być ograniczone czasowo i stosowane, gdy automatyzacja testów nie jest jeszcze dojrzała, lub jako ćwiczenie służące nauce i wykrywaniu ryzyka.

Śledzenie powiązań między historyjkami użytkownika, wymaganiami i kryteriami akceptacji oraz odpowiadającymi im przypadkami testowymi odgrywa kluczową rolę w testowaniu regresji. Umożliwia zespołom szybką identyfikację testów, które należy zaktualizować lub ponownie uruchomić po wprowadzeniu zmian, co ogranicza zbędny wysiłek i pozwala skoncentrować testowanie regresji na funkcjonalnościach, na które zmiany mają wpływ. Wskazuje również luki w pokryciu, pomagając zespołom w zrównoważeniu testów automatycznych, testów eksploracyjnych i testów regresji opartych na współpracy. Lekkie śledzenie powiązań (np. oznaczanie testów w oparciu o historyjki użytkownika w narzędziu do zarządzania backlogiem produktu) pozwala zespołom zachować pewność co do częstych wydań bez spowalniania dostaw.

2. Ludzie i zespoły (60 min.)

Słowa kluczowe

tester jednorazowy (ang. *tissue tester*), wytwarzanie sterowane testami akceptacyjnymi, wytwarzanie sterowane zachowaniem

Cele nauczania dla rozdziału 2

2.1 Podejście „cały zespół”

AT-2.1.1 (K2) Porównać generalizację i specjalizację w ramach zespołu.

AT-2.1.2 (K2) Podać przykłady motywowania interesariuszy biznesowych do udziału w testach.

AT-2.1.3 (K2) Podsumować, jak podejście „cały zespół” może wspomagać zespół wytwórczy.

2.2 Testerzy jednorazowi

AT-2.2.1 (K2) Wyjaśnić, jak i kiedy wykorzystywać testerów jednorazowych.

Wstęp

W projektach zwinnego wytwarzania oprogramowania ludzie i narzędzia stanowią podstawę szybkiego dostarczania wysokiej jakości. Podejście „cały zespół” podkreśla, że jakość jest wspólną odpowiedzialnością programistów, testerów i przedstawicieli jednostek biznesowych, którzy ściśle ze sobą współpracują. Zazwyczaj zespoły składają się z członków reprezentujących różne dyscypliny lub pełniących różne funkcje. Podczas gdy specjaliści wnoszą głęboką wiedzę ekspercką, generaliści dostosowują się do różnych ról, umożliwiając większą elastyczność i podział obciążenia pracą. Angażowanie i motywowanie przedstawicieli jednostek biznesowych do udziału w testowaniu poprzez wczesne włączenie ich w definiowanie kryteriów akceptacji, przeglądanie wyników testów oraz udział w testach eksploracyjnych poprawia jakość produktu i dostosowanie do celów biznesowych. Współpraca ta przynosi również korzyści programistom, ponieważ dzielenie się spostrzeżeniami z testów zmniejsza liczbę defektów i przyspiesza przekazywanie informacji zwrotnej. Zespoły zwinne mogą również angażować testerów jednorazowych – zewnętrznych testerów, którzy zapewniają „świeże spojrzenie” w celu wczesnego wykrywania defektów. Wiedza o tym, jak i kiedy korzystać z usług takich testerów, pomaga zapewnić zrównoważoną, obiektywną informację zwrotną bez spowalniania dostaw.

2.1. Podejście „cały zespół”

2.1.1. Generalizacja i specjalizacja w ramach zespołu

Istota podejścia „cały zespół” polega na tym, że testerzy, programiści i przedstawiciele jednostek biznesowych współpracują jako zespół zwinny na każdym etapie procesu rozwoju. Generalizacja i specjalizacja ról w zespole zwinnym mają bezpośredni wpływ na to, jak rozdzielane są czynności testowe, kto za nie odpowiada i jak są one udoskonalane. Metodyki zwinne promują podejście „cały zespół” do jakości, w którym każdy jest odpowiedzialny za jakość, niezależnie od tytułu czy stanowiska.

Testerzy zwinni są zazwyczaj specjalistami o kompetencjach typu T. Posiadają oni głęboką wiedzę specjalistyczną w zakresie testowania oraz szeroką wiedzę z wielu dziedzin, takich jak kodowanie, analiza biznesowa czy DevOps. Wspiera to współpracę międzyfunkcjonalną i umożliwia elastyczność w doborze składu zespołu.

Generalizacja pozwala wielu członkom zespołu wykonywać konkretne czynności. Wspiera to zrównoważone tempo pracy, zmniejsza wąskie gardła i zwiększa odporność zespołu. Zachęca również do wspólnej odpowiedzialności za jakość, co jest niezbędne w środowiskach ciągłego dostarczania. Specjalizacja ról nadal istnieje, szczególnie w przypadku zadań testowych wymagających konkretnej wiedzy specjalistycznej, takich jak techniki testowania, testowanie eksploracyjne, automatyzacja testów lub testowanie użyteczności. Jednak zespoły zwinne dążą do minimalizowania silosów, w których poszczególne osoby pracują w wąskich granicach ról, poprzez zachęcanie do dzielenia się wiedzą i pracy zespołowej.

Programowanie grupowe (ang. *mob programming*) i testowanie grupowe (patrz sekcja 5.2.1) zacierają podział ról w zespole. Podejścia te łączą cały zespół, w tym przedstawicieli jednostek biznesowych, programistów i testerów, w celu jednoczesnej pracy nad jednym zadaniem, zamiast ścisłego podziału pracy według wyspecjalizowanych ról.

Testerzy często przejmują inicjatywę, pomagając innym w stosowaniu praktyk zorientowanych na jakość, takich jak BDD, testowanie eksploracyjne i automatyzacja testów. Jednocześnie mogą uczyć się od programistów szczegółów implementacji, aby lepiej oceniać ryzyko i projektować testy. W ten sposób testerzy zachowują swoją specjalizację (głęboką wiedzę z zakresu

testowania), jednocześnie rozwijając szersze umiejętności, które przyczyniają się do wszechstronności całego zespołu.

Równowaga między specjalizacją a generalizacją powinna zależeć od kontekstu. Złożone dziedziny lub środowiska regulowane mogą wymagać bardziej wyspecjalizowanych ról, podczas gdy np. startupy mogą czerpać korzyści z szeroko wykwalifikowanych generalistów. Niezależnie od struktury, niezbędna jest jasna komunikacja i wzajemny szacunek wśród członków zespołu. Testerzy powinni angażować się w rozmowy od początku projektu, aby zapewnić wbudowanie jakości, a nie tylko jej późniejszą ocenę.

W środowiskach zwinnych z wieloma zespołami (np. SAFe, LeSS) podejście „cały zespół” wykracza poza poszczególne zespoły. Niezbędna staje się koordynacja między zespołami w zakresie standardów jakości, testów integracyjnych i odpowiedzialności za defekty na granicach zespołów. Testerzy często uczestniczą w społecznościach praktyków (ang. *communities of practice*), aby dzielić się wiedzą między zespołami (patrz (ISTQB-ATLAS)).

2.1.2. Motywowanie interesariuszy biznesowych do udziału w testach

Jakość jest wspólną odpowiedzialnością całego zespołu. Przedstawiciele jednostek biznesowych definiują jakość produktu, ale powinni również uczestniczyć w czynnościach testowych. Motywowanie do testowania zaczyna się od włączenia ich na wczesnym etapie procesu testowego, zwłaszcza w definiowaniu kryteriów akceptacji i formułowaniu testów zorientowanych na biznes. Ich zaangażowanie w te wczesne dyskusje gwarantuje, że dostarczone rozwiązanie niesie wartość dla klienta i jest zgodne z intencjami interesariuszy.

Testowanie nie powinno być traktowane jako odrębny etap, ale raczej jako wspólne, ciągłe działanie. Praktyki takie jak wytwarzanie sterowane testami akceptacyjnymi (ATDD), wytwarzanie sterowane zachowaniem (BDD) oraz specyfikacja przez przykłady (ang. *specification by example*) pomagają umocnić rolę przedstawiciela jednostki biznesowej w wyjaśnianiu zachowań poprzez konkretne, możliwe do przetestowania przykłady. Praktyki te sprawiają, że testowanie staje się widoczne i istotne z punktu widzenia interesów biznesowych, zachęcając przedstawicieli jednostek biznesowych do konstruktywnego dialogu z członkami zespołu i innymi interesariuszami.

Wbudowanie testowalności do historyjek użytkownika podczas dopracowywania backlogu produktu gwarantuje, że kryteria akceptacji są konkretne, a nie abstrakcyjne, powiązane z prawdziwymi, weryfikowalnymi wynikami testów. Ścisła współpraca z testerami i programistami podczas tych sesji pozwala przedstawicielom jednostek biznesowych dostrzec, jak niejasne wymagania prowadzą do niejednoznaczności i konieczności przeróbek, a dobrze sformułowane przykłady prowadzą do szybszego i bardziej przewidywalnego dostarczenia produktu.

Widoczność jest kluczowym czynnikiem motywującym. Tablice wskaźników (ang. *dashboards*) pokazujące pokrycie, trendy dotyczące defektów oraz metryki związane z ryzykami jakościowymi ułatwiają przedstawicielom jednostek biznesowych zrozumienie aktualnej jakości produktu oraz tego, gdzie testowanie wnosi wartość dodaną. Wskaźniki biznesowe, takie jak zadowolenie klientów, koszty wsparcia lub wdrażanie funkcji, pomagają przedstawicielom jednostek biznesowych postrzegać testowanie jako część walidacji produktu, a nie tylko weryfikacji. Te spostrzeżenia pomagają bezpośrednio powiązać testowanie z decyzjami dotyczącymi produktu i gotowością do wydania.

Wspólna odpowiedzialność jest wzmacniana w codziennej współpracy. Podczas planowania i dopracowywania iteracji przedstawiciele jednostek biznesowych powinni aktywnie pomagać w kształtowaniu tego, co oznacza „zrobione” dla konkretnej historyjki użytkownika, w tym pokrycia i kryteriów akceptacji. Ich obecność podczas działań testowych, takich jak ataki

na defekty (ang. *bug bashes*) czy testy użyteczności zapewnia, że pętle informacji zwrotnej pozostają krótkie i oparte na wartości biznesowej, a jednocześnie zwiększa ich motywację, ponieważ mogą na własne oczy zobaczyć owoce swojej pracy.

Wreszcie, uznanie wkładu przedstawicieli jednostek biznesowych w testowanie podczas retrospektyw i prezentacji (ang. *demos*) wzmacnia ich rolę w zapewnianiu jakości i utrzymuje ich zaangażowanie. Dzięki jasnym oczekiwaniom, dostępowi do odpowiednich informacji i ciągłym możliwościom wpływania na wyniki, przedstawiciele jednostek biznesowych są bardziej skłonni postrzegać testowanie jako kluczową część swoich obowiązków i potężne narzędzie umożliwiające dostarczanie udanych produktów.

2.1.3. Wsparcie programistów

Podejście „cały zespół” wspiera programistów poprzez włączenie współpracy ukierunkowanej na jakość w cały cykl wytwarzania oprogramowania. Ciągła interakcja między programistami, testerami i przedstawicielami jednostek biznesowych buduje wspólne zrozumienie celów jakościowych i potencjalnych ryzyk, co prowadzi do lepszych decyzji technicznych i biznesowych. Programiści czerpią korzyści z wczesnej informacji zwrotnej z testów, co zmniejsza koszty związane z defektami i wspiera szybkie działania naprawcze, zwłaszcza gdy stosowane są praktyki takie jak wytwarzanie sterowane testami akceptacyjnymi i wytwarzanie sterowane zachowaniem.

Testerzy ściśle współpracujący z programistami pomagają wyjaśnić kryteria akceptacji na podstawie przykładów, co prowadzi do ulepszonej specyfikacji przez przykłady i lepszego dostosowania kodu do oczekiwań biznesowych.

Testerzy mogą szkolić programistów w zakresie pisania lepszych testów modułowych i testów integracji modułów, aby osiągnąć wyższy poziom pokrycia kodu. Ponieważ testowanie staje się działaniem ciągłym, a nie procesem realizowanym na późnym etapie, programiści zyskują pewność siebie, aby bezpiecznie refaktoryzować i rozwijać bazę kodu. Automatyzacja testów na wczesnym etapie cyklu wytwarzania oprogramowania umożliwia szybsze wykrywanie defektów integracyjnych i ułatwia wdrażanie potoków CI/CD.

Ponadto podejście „cały zespół” przełamuje silosy ról, tworząc środowisko, w którym testerzy wnoszą wkład w przeglądy kodu z perspektywy jakości i ryzyka, a programiści uczestniczą w sesjach testów eksploracyjnych. To środowisko wzajemnego uczenia się nie tylko podnosi ogólne kompetencje zespołu w zakresie testowania, ale także sprzyja zbiorowej odpowiedzialności za jakość produktu. Programiści zyskują również wgląd w rzeczywiste zachowania użytkowników poprzez udział w testach eksploracyjnych, sesjach testów użyteczności lub analizę danych z produkcji, co pomaga im tworzyć oprogramowanie lepiej dostosowane do potrzeb użytkowników.

Innym sposobem, w jaki podejście „cały zespół” wspiera programistów, jest wczesne zaangażowanie działu wdrożeń i utrzymania, specjalistów ds. zabezpieczeń i testerów w celu uwzględnienia kwestii wdrożenia, monitorowania i zgodności na każdym etapie rozwoju. Ta współpraca pomaga programistom projektować rozwiązania, które są łatwiejsze do wdrożenia, obserwacji i zabezpieczenia w środowisku produkcyjnym, co ogranicza konieczność poprawek na późnych etapach oraz liczbę problemów operacyjnych. Dzięki uwzględnieniu tych perspektyw od samego początku programiści mogą dokonywać wyborów architektonicznych, które zapewniają równowagę między funkcjonalnością a cechami jakościowymi. Ostatecznie przyspiesza to dostarczanie produktu, jednocześnie zwiększając pewność co do gotowości systemu do rzeczywistego użytkowania.

2.2. Testerzy jednorazowi

Testerzy jednorazowi (ang. *tissue testers*) (znani również jako testerzy „świeżego spojrzenia”) to tymczasowi testerzy spoza zespołu, którzy zapewniają szybką, nieformalną informację zwrotną na temat produktu lub funkcji w fazie rozwoju. Jest to zazwyczaj stosowane w sektorach charakteryzujących się dużą zmiennością, takich jak branża gier wideo. Są oni najskuteczniejsi, gdy wykorzystuje się ich we wczesnych fazach projektowania lub wdrażania funkcji, zanim zespoły podejmą kosztowne decyzje dotyczące rozwoju lub infrastruktury. Chodzi o to, żeby na chwilę pokazać nowy pomysł, interfejs lub interakcję komuś, kto nie jest z tym zaznajomiony, żeby wykryć wady użyteczności, mylące zachowania lub wprowadzające w błąd przebiegi.

Testerami jednorazowymi są zazwyczaj osoby spoza zespołu lub projektu (na przykład współpracownik z innego zespołu) i należy z nich korzystać ze świadomością, że ich opinie mogą się znacznie różnić w zależności od osoby, a ich zaangażowanie ma charakter krótkotrwały. Po pierwszym obejrzeniu produktu nie są już naprawdę bezstronni, więc ich wartość jako wnoszących „świeże spojrzenie” maleje (patrz sekcja 4.1.4). Metafora jednorazowości podkreśla lekki, przejściowy charakter tej praktyki. Nie należy ponownie korzystać z usług danego testera jednorazowego, gdy już zapoznał się z testowanym projektem, ponieważ na jego reakcje będzie miała wpływ wcześniejsza wiedza. W tym momencie traci on niezbędną, świeżą perspektywę.

Testerzy jednorazowi najlepiej sprawdzają się w testowaniu eksploracyjnym, testowaniu użyteczności lub weryfikacji założeń, gdzie szybka informacja zwrotna jest cenna, ale jest za wcześnie na rygorystyczną walidację. Na przykład, przed dopracowaniem interakcji lub przebiegu przez interfejs na ekranie, programista lub tester może przeprowadzić testera jednorazowego przez ten proces, aby zidentyfikować potencjalne problemy. Ważne jest, aby nie wyjaśniać zbyt wiele i tym samym nie wprowadzać stronniczości, ponieważ zniszczy to ich świeże spojrzenie na produkt.

Testy jednorazowe nie powinny zastępować testów ustrukturyzowanych. Powinny raczej je poprzedzać lub uzupełniać. Ich główna wartość polega na wczesnym wykrywaniu rozbieżności między zamierzeniami projektowymi a faktycznym zrozumieniem użytkownika. Ponieważ testy jednorazowe są szybkie w przeprowadzeniu i nie wymagają rekrutacji zewnętrznych użytkowników, są one przydatne w kontekście zwinnego wytwarzania oprogramowania i DevOps.

3. Zarządzanie testami i doskonalenie procesu testowego (210 min.)

Słowa kluczowe

doskonalenie procesu testowego, kwadranty testowe, monitorowanie testów, nadzór and testami, planowanie testów, raportowanie testów, strategia testów, zwinne wytwarzanie oprogramowania

Słowa kluczowe specyficzne dla dziedziny

Cel-Pytanie-Metryka, ciche wdrożenie (ang. *dark launch*), definicja ukończenia, metryka, przełącznik funkcji, społeczność praktyków (ang. *community of practice*), wydanie kanarkowe (ang. *canary release*)

Cele nauczania dla rozdziału 3

3.1 Planowanie testów

AT-3.1.1 (K2) Podsumować planowanie testów w zwinnym wytwarzaniu oprogramowania.

AT-3.1.2 (K4) Nakreślić projektową strategię testową używając kwadrantów testowych.

3.2 Monitorowanie testów i nadzór nad testami

AT-3.2.1 (K2) Wyjaśnić, jak monitorować testy i sprawować nadzór and testami w zwinnym wytwarzaniu oprogramowania.

3.3 Raportowanie testów

AT-3.3.1 (K2) Porównać różne typy pokrycia, które można użyć do raportowania testów w zwinnym wytwarzaniu oprogramowania.

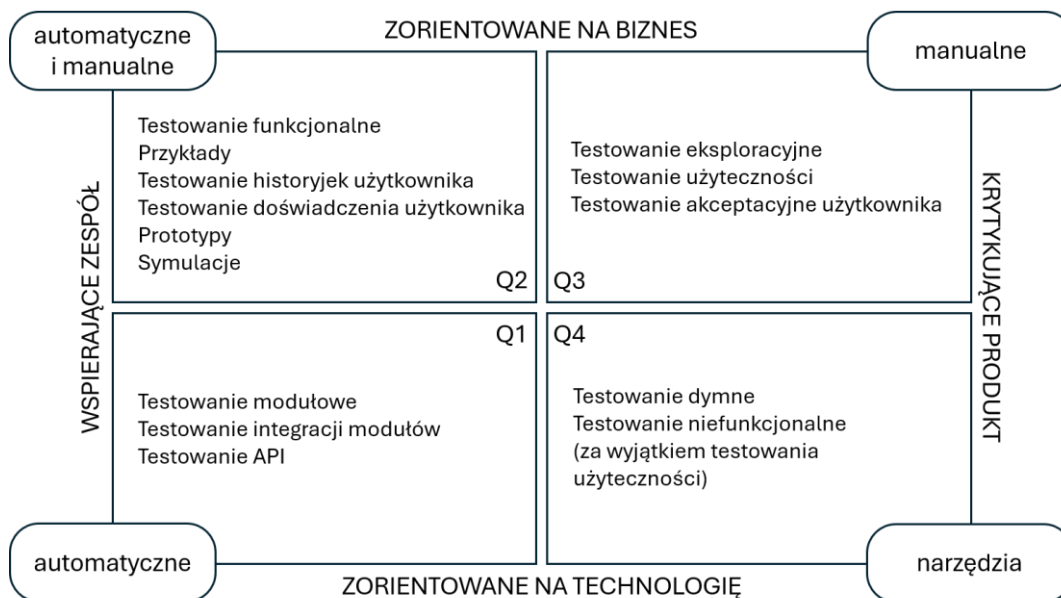
3.4 Doskonalenie procesu testowego

AT-3.4.1 (K4) Wybrać odpowiednie działania mające na celu doskonalenie procesu testowego w oparciu o wskaźniki stosowane w zwinnym wytwarzaniu oprogramowania.

AT-3.4.2 (K2) Wyjaśnić, jak przeprowadzić udoskonalenie procesu testowego w zwinnym wytwarzaniu oprogramowania.

Wstęp

W projektach zwinnego wytwarzania oprogramowania czynności testowe są zintegrowane z krótkimi, iteracyjnymi cyklami testowymi, co wymaga adaptacyjnego planowania testów, które ewoluuje wraz z priorytetami produktu i zespołu. Projektowa strategia testowa jest wspierana przez kwadranty testowe (Gregory i Crispin, 2019), które służą do wyboru odpowiednich typów testów, poziomów testów, podejść do testów i technik testowania w celu osiągnięcia zrównoważonego pokrycia potrzeb biznesowych i technologicznych.



Rys. 1. Model kwadrantów testowych

Kwadrant Q1 zawiera testy zorientowane na technologię, które wspierają zespół, pomagając kierować rozwojem. Weryfikują one niewielkie fragmenty kodu. Kwadrant Q2 zawiera testy zorientowane na biznes, które wspierają zespół. Weryfikują one, czy produkt zachowuje się zgodnie z oczekiwaniami biznesowymi. Kwadrant Q3 zawiera testy zorientowane na biznes, które walidują (krytykują) produkt. Sprawdzają one, jak dobrze produkt spełnia potrzeby użytkowników. Kwadrant Q4 zawiera testy zorientowane na technologię, które walidują (krytykują) produkt. Weryfikują one charakterystyki jakościowe produktu.

Zasady Agile dodatkowo kształtują strategię testów poprzez promowanie planowania adaptacyjnego, wczesnej informacji zwrotnej oraz ciągłego ustalania priorytetów w oparciu o ryzyko, aby dostosować testowanie do wartości biznesowej. Monitorowanie testów i nadzór nad testami odbywają się na bieżąco, z wykorzystaniem lekkich metryk mierzonych w czasie rzeczywistym do śledzenia postępów, wykrywania ryzyk i dostosowywania priorytetów. Raportowanie testów dostarcza znaczących, opartych na kontekście metryk pokrycia, takich jak pokrycie wymagań, kodu i ryzyka, aby zapewnić interesariuszom proste i jednoznaczne informacje na temat stanu jakości. Ciągłe doskonalenie procesu testowego jest wpisane w metodologię Agile, wykorzystując metryki do identyfikacji wąskich gardeł, udoskonalania praktyk i zwiększania wydajności. Dzięki wczesnemu i częstemu wprowadzaniu udoskołań zespoły mogą reagować na zmiany, utrzymywać jakość i maksymalizować wartość testowania w ramach każdej iteracji.

3.1. Planowanie testów

3.1.1. Planowanie testów w zwinnym wytwarzaniu oprogramowania

W metodykach zwinnych planowanie testów zazwyczaj odbywa się na dwóch głównych poziomach: planowania iteracji i planowania wydania. Projektowa strategia testowa ma wpływ na te działania planistyczne.

Planowanie iteracji

Podczas planowania iteracji zespół przegląda backlog produktu i wybiera historyjki użytkownika do danej iteracji. Testerzy mogą wnieść swój wkład, organizując sesje burzy mózgów z przedstawicielami jednostek biznesowych i programistami w celu zidentyfikowania ryzyk i ustalenia priorytetów czynności testowych, zdefiniowania wysokopoziomowych warunków testowych oraz dopracowania kryteriów akceptacji.

Planowanie iteracji obejmuje wybór odpowiednich typów testów, poziomów testów, podejść do testów i technik testowania dla danej iteracji. Testerzy wykorzystują swoją wiedzę i doświadczenie, aby pomóc zespołowi w dokonywaniu właściwych wyborów. Do wsparcia tego zadania można wykorzystać kwadranty testowe.

Planowanie iteracji musi uwzględniać środowisko testowe i potrzeby w zakresie danych testowych, aby osiągnąć gotowość do iteracji. Ponieważ w projektach zwinnych testowanie nie jest oddzielną fazą i może mieć miejsce w dowolnym momencie iteracji, środowiska testowe i dane testowe powinny być stabilne i dostępne od początku iteracji. Plan testów zazwyczaj obejmuje praktyki ciągłej integracji i ciągłego testowania. Testowanie manualne uzupełnia automatyzację testów, zajmując się obszarami wymagającymi ludzkiej oceny, takimi jak testy użyteczności i testy eksploracyjne.

Planowanie iteracji obejmuje uzgodnienie lub ponowne przeanalizowanie definicji ukończenia na poziomie zespołu, szczegółowe określenie kryteriów wyjścia dla historyjek użytkownika i innych elementów backlogu w ramach iteracji. Wyjaśnia również, w jaki sposób wykonanie testów, wyniki testów i informacje o osiągniętym pokryciu przyczyniają się do spełnienia definicji ukończenia dla każdego elementu i dla wyniku iteracji.

Planowanie wydania

Podczas planowania wydania planowanie testów odbywa się na szerszym poziomie niż w przypadku planowania iteracji. Zespół identyfikuje wysokopoziomowe cele biznesowe i ocenia backlog produktu w celu określenia zakresu i kolejności dostarczanych funkcji. Z punktu widzenia testowania obejmuje to ocenę ryzyk jakościowych obejmujących cały zakres wydania, zdefiniowanie celów testów na poziomie wydania oraz określenie odpowiednich poziomów testów, typów testów, podejść do testów i technik testowania niezbędnych do łagodzenia tych ryzyk.

Testerzy wnoszą swój wkład poprzez identyfikację zależności między funkcjami, ograniczeń dotyczących środowisk testowych i danych oraz punktów integracji, które wymagają szczególnej walidacji. Pomagają oni oszacować ogólny nakład pracy związany z testowaniem przy użyciu technik opartych na metrykach i wiedzy eksperckiej, co wspiera zdolność zespołu do planowania wydajności zespołu w wielu iteracjach.

Zespół opracowuje strategię testów na poziomie wydania, która obejmuje co najmniej testy funkcjonalne i niefunkcjonalne, sposób ich włączenia do poszczególnych iteracji oraz sposób zarządzania testami regresji. Zapewnia to wczesne planowanie testów opartych na doświadczeniu i pozwala zidentyfikować obszary, w których konieczne jest zaangażowanie

zewnętrznych interesariuszy, takich jak klienci lub eksperci ds. zgodności. Tester, często pełniący rolę moderatora, może poprowadzić zespół przez ćwiczenia oparte na współpracy (np. warsztaty z mapowaniem myśli lub sesje burzy mózgów dotyczące ryzyka) w celu opracowania strategii.

Planowanie wydania obejmuje uzgodnienie definicji ukończenia (ang. *Definition of Done*) na poziomie wydania, w tym kryteriów akceptacji dla zintegrowanego produktu. Zespół określa również, w jaki sposób wyniki testów, takie jak informacje o pokryciu, będą weryfikowane i przekazywane w celu wsparcia oceny gotowości do wydania.

3.1.2. Projektowa strategia testowa

Projektowa strategia testowa jest kształtowana przez kilka czynników kontekstowych, z których każdy wpływa na to, w jaki sposób poziomy testów, typy testów, podejścia do testów i techniki testowania są przypisywane do kwadrantów testowych. Złożoność dziedziny produktu i jej znaczenie dla działalności biznesowej stanowią podstawę tej strategii. W dziedzinach mających konsekwencje prawne lub związane z bezpieczeństwem (np. finanse, opieka zdrowotna, lotnictwo) kładzie się większy nacisk na metody z kwadrantów Q2 i Q4, gdzie preferowane są identyfikowalne, ustrukturyzowane podejścia zapewniające audytowalne dowody zgodności z przepisami i odpowiednie pokrycie (np. testy нефункционалне, takie jak testy wydajności, testy zabezpieczeń, testy kompatybilności lub testy niezawodności).

Struktura organizacyjna i skład zespołu

Wpływają one na głębokość i zakres testowania. Zespół wielofunkcyjny posiadający specjalistów ds. testów w zakresie testowania doświadczenia użytkownika (UX), zabezpieczeń lub wydajności umożliwia głębsze testowanie we wszystkich kwadrantach. Natomiast zespoły, które polegają na zewnętrznych lub odizolowanych ekspertach, mogą niedostatecznie wykorzystywać testy eksploracyjne i testy нефункционалне, chyba że testy te są wyraźnie zaplanowane i wspierane. Podobnie, gdy zespoły przyjmują silną postawę wspólnej odpowiedzialności za jakość, techniki testowania oparte na doświadczeniu, takie jak testy eksploracyjne z kwadrantu Q3, są stosowane w sposób ciągły i zintegrowane z procesem rozwoju, a nie ograniczone do końca iteracji.

Częstotliwość dostaw i strategia wydawania

Określają one, ile czasu jest dostępne na testowanie i mają bezpośredni wpływ na stopień automatyzacji testów. Krótkie cykle wydawania i modele ciągłego dostarczania wymagają szerokiej automatyzacji testów w kwadrantach Q1 i Q4. W tym przypadku wybór metod skłania się ku łatwym w utrzymaniu, niedrogim opcjom informacji zwrotnej (np. testowanie modułowe, testowanie API, testy dymne). Z drugiej strony, w przypadku wydawania oprogramowania w dłuższych cyklach lub opartych na kamieniach milowych, może być więcej okazji do testowania manualnego w kwadrancie Q3 (np. testy eksploracyjne, dogłębne testy użyteczności i testy akceptacyjne użytkownika).

Obserwowalność

Jeśli obserwowalność jest niska, trudno jest polegać na domyślnym, biernym monitorowaniu, co zwiększa zapotrzebowanie na elementy z kwadrantu Q2, takie jak symulacje i prototypy. Jeśli testowalność jest wysoka, zespoły mogą wprowadzić oprzyrządowanie i interfejsy przeznaczone wyłącznie do testowania, aby umożliwić bardziej ukierunkowane testowanie we wszystkich kwadrantach.

Dojrzałość produktu i częstotliwość zmian

W przypadku produktów lub systemów na wczesnym etapie rozwoju, poddawanych szybkiemu prototypowaniu, strategia testów zazwyczaj opiera się na mechanizmach informacji zwrotnej

o niskim nakładzie i wysokiej wartości. Testowanie eksploracyjne z kwadrantu Q3 ma pierwszeństwo przed formalnym modelowaniem lub wyczerpującym pokryciem. W miarę dojrzewania produktu wzrasta ryzyko regresji, co wymaga większych inwestycji w łatwą w utrzymaniu automatyzację testów i testowanie strukturalne (np. testy modułowe z kwadrantu Q1 lub testy funkcjonalne z kwadrantu Q2 zaprojektowane z wykorzystaniem czarnoskrzynkowych technik testowania), aby złagodzić rosnące koszty zmian. Kwadranty Q1 i Q2 w coraz większym stopniu opierają się na technikach testowania opartych na pokryciu i automatyzacji testów sterowanej danymi, aby utrzymać tempo.

Ograniczenia narzędziowe i dojrzałość infrastruktury

Tam, gdzie dostępne są nowoczesne potoki CI/CD, obiekty imitacji (ang. *test doubles*) i środowiska kontenerowe, automatyzację testów i wirtualizację można w pełni wykorzystać przy użyciu kwadrantów Q1 i Q2, a narzędzia umożliwiają również stosowanie testów niefunkcyjnych z kwadrantu Q4. Bez tych możliwości strategia testów może ograniczać się do testów manualnych (np. testów eksploracyjnych lub testów akceptacyjnych użytkownika z kwadrantu Q3) lub opierać się na izolowanych środowiskach testowych. Wiąże się to z ryzykiem i wymaga środków kompensacyjnych, takich jak zwiększony zasięg testów eksploracyjnych i przełączniki funkcji (ang. *feature toggles*), aby zarządzać niepewnością w środowisku produkcyjnym.

Skłonność do ryzyka i kultura przywództwa

Organizacja unikająca ryzyka lub kierująca się przepisami może egzekwować podejścia do testów oparte na złożonych procesach, priorytetowo traktując udokumentowane pokrycie, rozbudowane warunki testowe i metryki ilościowe. Zespoły w takich kontekstach mają tendencję do stosowania czarnoskrzynkowych technik testowania do testów funkcjonalnych z kwadrantu Q2 (np. analiza wartości brzegowych, testowanie przejść pomiędzy stanami, testowanie tablicy decyzyjnej). Kultura, która ceni adaptacyjne uczenie się i eksperymentowanie, wspiera testy eksploracyjne i testy akceptacyjne użytkownika z kwadrantu Q3, a także symulacje z kwadrantu Q2. Skuteczne testowanie wymaga zrównoważenia wszystkich aspektów – kwadranty Q1 i Q2 zazwyczaj zapewniają powtarzalność i pewność, podczas gdy kwadranty Q3 i Q4 sprzyjają innowacjom i uczeniu się.

Mechanizmy zbierania opinii klientów i obserwowalność produkcji

Mogą one rozszerzyć strategię testów poza etap dostawy. Podejścia do testów, takie jak testowanie A/B, telemetria użytkownika przez klientów oraz analiza wzorców awarii, wspierają ciągłą walidację wartości biznesowej i tolerowania usterek operacyjnych. Te praktyki testowe rozszerzają kwadranty Q3 i Q4 na środowiska produkcyjne, umożliwiając krótkie pętle informacji zwrotnej.

Strategię testów należy okresowo weryfikować, np. w oparciu o wyniki retrospektyw. W miarę ewolucji produktu i jego kontekstu zakres strategii testów powinien być dostosowywany poprzez przesuwanie się między kwadrantami, nieustannie równoważąc proporcje testów skryptowych i swobodnych, a także testów manualnych i automatycznych. Im bardziej dynamiczny i ciągły jest model dostarczania, tym częściej konieczna staje się taka ponowna ocena.

3.2. Monitorowanie testów i nadzór nad testami

W zwinnym wytwarzaniu oprogramowania monitorowanie testów wykorzystuje lekkie metryki wyświetlane za pomocą wykresów spalania, diagramów przepływu skumulowanego, automatycznych tablic wskaźników i podobnych wizualizacji w celu ciągłego śledzenia postępów testów i jakości produktu. Wizualizacje te są często aktualizowane i dostępne dla wszystkich członków zespołu, aby umożliwić wspólną świadomość sytuacji i natychmiastowe działanie

w przypadku wystąpienia odchyień. Zazwyczaj metryki koncentrują się na dostarczanej wartości i zapewnianiu praktycznej informacji zwrotnej w czasie rzeczywistym, a nie tylko na okresowym raportowaniu. Nacisk kładziony jest na zrozumienie trendów w wykrywaniu defektów, pokryciu automatyzacji testów, wyników testów eksploracyjnych oraz danych z monitorowania produkcji, a nie na śledzenie zgodności z wcześniej zdefiniowanymi planami testów.

W zespołach zwinnych nadzór nad testami jest sprawowany wspólnie podczas codziennych spotkań na stojąco (ang. *daily stand-ups*), przeglądów iteracji i retrospektyw, gdzie decyzje opierają się na danych z potoków CI/CD. Korekty są wprowadzane w czasie rzeczywistym, często na poziomie historyjek użytkownika lub funkcji, w tym poprzez ustalanie priorytetów sesji testów eksploracyjnych, dopracowywanie kryteriów akceptacji oraz tymczasowe skupianie się na obszarach wysokiego ryzyka. Działania te są realizowane przez cały zespół, a nie tylko przez testerów, co gwarantuje, że zmiany zakresu, harmonogramu i celów jakościowych są natychmiast omawiane i uzgadniane. Informacje zwrotne z automatyzacji testów, sygnały z monitorowania produkcji oraz pętla informacji zwrotnej od klientów są traktowane jako kluczowe czynniki wyzwalające decyzje w ramach nadzoru nad testami.

Monitorowanie testów i nadzór nad testami powinny być zintegrowane z codziennym przepływem pracy, z wykorzystaniem współpracy międzyfunkcjonalnej i ciągłej informacji zwrotnej zarówno ze środowisk przedprodukcyjnych, jak i produkcyjnych. Decyzje są często ponownie oceniane w oparciu o zmieniające się priorytety biznesowe i aktualny stan dostarczonych przyrostów, co wspiera zasadę zwinnego wytwarzania oprogramowania polegającą na szybkim reagowaniu na zmiany przy jednoczesnej ochronie celów jakościowych.

3.3. Raportowanie testów

Raportowanie testów obejmuje różne działania, ale ta sekcja skupia się na różnych metrykach pokrycia, które odzwierciedlają postęp testów, wspierają podejmowanie decyzji i zwiększają przejrzystość. Używane rodzaje pokrycia powinny być zgodne z celami zespołu, wspierać szybkie cykle informacji zwrotnej i być na tyle proste, aby były przydatne bez powodowania niepotrzebnych obciążeń. Zespoły zwinne preferują metryki pokrycia, które sprzyjają wspólnemu zrozumieniu jakości systemu i ryzyk.

Pokrycie wymagań jest powszechne, zwłaszcza w kontekście ATDD lub BDD. Łącząc zautomatyzowane testy akceptacyjne z historyjkami użytkownika lub przykładami, zespoły mogą wykazać, które aspekty funkcjonalne zostały określone, opracowane i zweryfikowane. Te kryteria akceptacji stają się identyfikowalnymi elementami pokrycia, pomagając zespołowi w komunikowaniu gotowości i kompletności historyjek użytkownika.

Pokrycie kodu może być stosowane wybiórczo, zwłaszcza w wysoce zautomatyzowanych środowiskach testowych. Należy je interpretować ostrożnie i nie traktować jako wskaźnika jakości. Chociaż wysokie pokrycie kodu jest przydatne do identyfikacji nieprzetestowanych ścieżek logicznych, samo w sobie nie gwarantuje znaczącego testowania. Zespoły zwinne mogą łączyć analizę pokrycia z posiewem usterek (ang. *fault seeding*) lub przeglądami jakości asercji, aby uzyskać głębszy wgląd w skuteczność testów.

Pokrycie związane z testowaniem eksploracyjnym można raportować za pomocą zarządzania testowaniem w sesjach lub wskaźników realizacji karty opisu testu. Jest to szczególnie cenne w przypadku testów eksploracyjnych zorientowanych na biznes, testów użyteczności lub testów w obszarach ryzyka, w których brakuje formalnych specyfikacji. Wizualne tablice wskaźników mogą służyć do przedstawienia tego, co zostało zbadane oraz które scenariusze lub osoby zostały przetestowane.

Pokrycie infrastruktury i środowiska, szczególnie w kontekście DevOps i ciągłego dostarczania, może być śledzone w celu pokazania, które platformy, konfiguracje lub interfejsy API zostały przetestowane. Narzędzia do monitorowania i obserwowalności również usprawniają raportowanie pokrycia operacyjnego, ujawniając wzorce użytkowania i anomalie w środowisku produkcyjnym.

We wszystkich przypadkach metryki pokrycia w zwinnym wytwarzaniu oprogramowania powinny być omawiane wspólnie, a nie traktowane wyłącznie jako abstrakcyjne wskaźniki. Muszą one wspierać świadome podejmowanie decyzji i być dostosowane do kontekstu organizacyjnego, projektowego i zespołowego, a nie narzucane jako wskaźniki nakazowe. Metryki testowe są narzędziami komunikacji i doskonalenia w iteracyjnych i adaptacyjnych cyklach wytwarzania oprogramowania. Zespoły powinny opierać się iluzji kompletności tworzonej przez metryki i zamiast tego dostarczać informacje dostosowane do potrzeb różnych interesariuszy.

3.4. Doskonalenie procesu testowego

3.4.1. Metryki w doskonaleniu procesu testowego

Doskonalenie procesu testowego oparte na metrykach w zwinnym wytwarzaniu oprogramowania wymaga starannej interpretacji zarówno wskaźników produktu, jak i procesu, zgodnie z zasadami ciągłego doskonalenia oraz z wykorzystaniem możliwości zespołu i narzędzi. Z wyjątkiem rzeczywistych kluczowych wskaźników wydajności (ang. *key performance indicators*), wskaźniki powinny służyć jako mechanizmy informacji zwrotnej, a nie jako cele wydajnościowe, aby uniknąć nadużyć, które mogłyby zniekształcić zachowania i prowadzić do lokalnej optymalizacji.

Gdy odsetek wykrytych defektów nieoczekiwanie spada, analiza podstawowej przyczyny może ujawnić nieskuteczne testy eksploracyjne lub niewystarczającą automatyzację testów. W odpowiedzi zespół może rozważyć udoskonalenie technik tworzenia kart opisu testu, poprawę zarządzania testowaniem w sesjach lub inwestycję w narzędzia wspierające obserwowalność lub testy eksploracyjne.

Jeśli testy wielokrotnie kończą się niepowodzeniem z powodu niestabilności środowiska, inwestycja w zarządzanie środowiskiem testowym, wirtualizację usług lub rozwiązania kontenerowe może poprawić stabilność i zaufanie do wyników testów.

Długie cykle rozwiązywania defektów mogą wskazywać na słabą współpracę między rolami lub silosowy podział obowiązków. Możliwym udoskonaleniem może być przyjęcie podejścia „cały zespół” do klasyfikacji defektów, włączenie testerów na wcześniejszym etapie sesji udoskonalania oraz włączenie liderów testów zwinnych w celu wzmocnienia wspólnej odpowiedzialności za jakość. Połączenie metryk takich jak średni czas do awarii i średni czas naprawy z retrospektywami pozwala zespołom rozpoznać wąskie gardła w pętlach informacji zwrotnej i odpowiednio się dostosować.

Niski poziom pokrycia na ścieżkach krytycznych, wykryty przez narzędzia do pomiaru pokrycia kodu, może skłonić do podjęcia działań mających na celu zwiększenie automatycznego pokrycia na poziomie testów modułowych lub testów integracji modułów, co potencjalnie może spowodować przesunięcie równowagi w piramidzie testów w kierunku bardziej szczegółowych i szybkich testów. Zespoły muszą jednak ocenić, czy testy te dostarczają istotnych informacji, czy też jedynie zawyżają wartości metryk.

Redundantne lub niestabilne testy, ilustrowane wysokimi wskaźnikami niepowodzeń przy niskiej liczbie wykrytych defektów, powinny zostać zrefaktoryzowane lub zastąpione przy użyciu wzorców projektowych, takich jak *Page Object Pattern* czy *Test Data Builder*. Więcej informacji na temat wzorców projektowych w automatyzacji testów można znaleźć w (ISTQB-TAE, w. 2.0).

Spadający udział w retrospektywach lub dyskusjach dotyczących jakości może odzwierciedlać problem kulturowy. Może być konieczny coaching, aby zapewnić poczucie bezpieczeństwa psychicznego i wzmocnić postrzeganie testowania jako wspólnej odpowiedzialności. W takim kontekście metryki jakościowe, takie jak postrzegane zaufanie i otwartość we współpracy, mogą być równie ważne jak metryki ilościowe.

Gdy narasta dług testowy, o czym świadczy rosnący backlog testów niestabilnych (ang. *flaky tests*) lub czas wykonania manualnych testów regresji, strategia testów może przewidywać w każdej iteracji czas na utrzymanie testów, rotację odpowiedzialności za testy oraz stosowanie praktyk refaktoryzacji w automatyzacji testów. Pomiar skuteczności automatyzacji testów powinien obejmować nie tylko wskaźniki zaliczonych i niezaliczonych wyników, ale także nakład pracy związany z utrzymaniem oraz czas realizacji testów (ang. *test execution lead time*).

Metryki oparte na monitorowaniu, takie jak średni czas do naprawy (ang. *mean time to recovery*, MTTR) lub wskaźnik defektów przedostających się do produkcji, mogą zainicjować udoskonalenia procesu testowego. Wysoki MTTR może wskazywać na brakujące alerty systemowe, ograniczenia zasobów lub niewystarczające testy eksploracyjne zachowań operacyjnych. Może to prowadzić do włączenia inżynierii chaosu (ang. *chaos engineering*, *chaos testing*) lub technik rozwoju oprogramowania, takich jak wydania kanarkowe, przetłaczanie funkcji lub ciche wdrożenia (ang. *dark launches*).

W przypadku rozwoju pracowników metryki takie jak częstotliwość testowania w parach, liczba sesji poświęconych nauce o defektach lub wskaźniki wkładu międzyfunkcjonalnego mogą wskazywać, czy ma miejsce dzielenie się wiedzą i wzrost kompetencji. Brak współpracy międzyfunkcjonalnej między rolami może wymagać mentoringu, rotacji ról lub ustrukturyzowanych ścieżek uczenia się, takich jak inicjatywy społeczności praktyków (ang. *community of practice*).

Udoskonalenia związane z narzędziami mogą opierać się na metrykach czasu realizacji od zatwierdzenia kodu do informacji zwrotnej, analizie testów niestabilnych oraz wskaźnikach niepowodzeń integracji. Narzędzia umożliwiające krótkie pętle informacji zwrotnej, takie jak tablice wskaźników, automatyczna analiza podstawowych przyczyn lub inteligentny wybór testów, mogą bezpośrednio wspierać uczenie się zespołu i umożliwiać szybsze reakcje.

Wszelkie działania doskonalące powinny być mierzalne i weryfikowane. W tym zakresie istotną rolę mogą odgrywać retrospektywy. Metryki muszą być powiązane z celami testów i priorytetami ryzyka, a nie z arbitralnymi punktami odniesienia. Modele takie jak GQM (Cel-Pytanie-Metryka, ang. *Goal-Question-Metric*) są pomocne w strukturyzowaniu wykorzystania metryk w sposób zorientowany na cele. Wreszcie, zespoły powinny zachować krytyczne myślenie i zdolność do rozumienia kontekstu, aby zapobiec oderwaniu metryk od podstawowej misji testowej.

3.4.2. Doskonalenie procesu testowego w zwinnym wytwarzaniu oprogramowania

W celu doskonalenia procesu testowego w zwinnym wytwarzaniu oprogramowania zespoły mogą przyjąć ustrukturyzowane, ale lekkie podejścia, które kładą nacisk na współpracę, zdolność do rozumienia kontekstu i iteracyjne eksperymentowanie:

Przeprowadzaj ukierunkowane retrospektywy. Stosuj lekkie, oparte na współpracy praktyki, takie jak retrospektywy poświęcone testom, aby zidentyfikować wyzwania związane z testowaniem i możliwości udoskonalień. Działania doskonalące muszą być osadzone w aktualnym kontekście zespołu, a nie narzucone z zewnątrz. Retrospektywy skupione konkretnie na testowaniu, odrębne od ogólnych retrospektyw, mogą zapewnić ukierunkowany wgląd

w skuteczność praktyk testowych i ich zgodność z potrzebami biznesowymi i technicznymi (ISTQB-ATLAS, w. 1.0) oraz pomóc w budowaniu wspólnej odpowiedzialności za cele jakościowe.

Ustal ocenę bazową. Przeprowadź ustrukturyzowane oceny testów zwinnych, aby zidentyfikować luki w obecnym podejściu do testów. Narzędzia takie jak samooceny lub listy kontrolne mogą skłonić do refleksji nad podstawowymi wymiarami testowania zwinnego, takimi jak współpraca, pętla informacji zwrotnej i ciągła integracja. Na tej podstawie ukierunkowane udoskonalenia mogą przebiegać stopniowo i iteracyjnie.

Dostosuj działania poprzez wspólne warsztaty ryzyka. Zaproś przedstawicieli jednostek biznesowych, testerów i programistów, aby zbudować wspólne zrozumienie krytycznych ryzyk produktowych oraz obszarów, na których najlepiej skoncentrować wysiłki testowe. Zapewni to, że udoskonalenia będą zgodne zarówno z celami biznesowymi, jak i z celami zespołu.

Wzmocnij pętlę informacji zwrotnej CI/CD. Zintegruj praktyki, takie jak podejście „najpierw test” (ang. *test first*) i testowanie w parach, aby umożliwić wczesną, praktyczną informację zwrotną. Wykorzystaj testowanie eksploracyjne nie tylko do wykrywania defektów, ale także do odkrywania nieznanych ryzyk i luk w testowaniu, wspierając adaptacyjne usprawnianie procesu testowego.

Przeprowadzaj małe eksperymenty. Wprowadź techniki, takie jak zarządzanie testowaniem w sesjach, aby stopniowo badać zmiany w procesie testowym. Wyniki tych eksperymentów powinny być otwarcie omawiane w zespole, a retrospektywy powinny służyć do dostosowywania strategii testów w razie potrzeby.

Wykorzystuj widoczne metryki z rozważą. Monitoruj stan automatyzacji testów, trendy dotyczące defektów i pokrycie w ramach potoków CI/CD, ale powstrzymaj się od pokusy porównywania zespołów między sobą. Zamiast tego skup się na metrykach, które pomagają zespołom zrozumieć ich własną skuteczność i zwiększyć widoczność ich pracy.

Zespoły mogą korzystać z praktyk takich jak mapowanie strumienia wartości (ang. *Value Stream Mapping*), cykl PDCA (ang. *Plan-Do-Check-Act*) lub diagramy pętli przyczynowo-skutkowych (ang. *casual loop diagrams*) (patrz (ISTQB-ATLAS, w. 1.0)). Udoskonalenia muszą uwzględniać czynniki socjotechniczne, takie jak kultura organizacyjna i praktyki komunikacyjne. Opór przed zmianami należy łagodzić poprzez rozmowy sprzyjające integracji oparte na strategiach ułatwiających wprowadzanie zmian (Prosci, 2026).

Ciągłe doskonalenie w testowaniu zwinnym musi być zakorzenione w krytycznym myśleniu, ciągłym uczeniu się i współpracy całego zespołu. Stosowanie ustalonych modeli lub procesów powinno być starannie dostosowane do kontekstu. Każde udoskonalenie powinno pełnić rolę hipotezy, którą należy przetestować poprzez działanie, uczenie się i adaptację.

4. Przesunięcie w lewo (135 min.)

Słowa kluczowe

jakość, podstawa testów, przesunięcie w lewo, testalia, *testboarding*, wytwarzanie sterowane testami akceptacyjnymi, wytwarzanie sterowane zachowaniem

Słowa kluczowe specyficzne dla dziedziny

arkusz sesji testowej, historyjka użytkownika, mapowanie przykładów (ang. *example mapping*), ścieżka użytkownika (ang. *user journey*), podział historyjki użytkownika (ang. *user story slicing*), specyfikacja przez przykłady (ang. *specification by example*), *storyboarding*, ustrukturyzowana rozmowa (ang. *structured conversation*), stronniczość (ang. *bias*)

Cele nauczania dla rozdziału 4

4.1 Użycie przesunięcia w lewo w celu zwiększenia jakości podstawy testów

AT-4.1.1 (K2) Podać przykłady tego, jak testalia mogą być użyte jako forma wymagań.

AT-4.1.2 (K2) Wyjaśnić, jak *storyboarding* i *testboarding* mogą być wykorzystane do zwiększenia jakości podstawy testów.

AT-4.1.3 (K2) Wyjaśnić, jak mapowanie przykładów może być wykorzystane do zwiększenia jakości podstawy testów.

AT-4.1.4 (K2) Podać przykłady tego, jak stronniczości mogą negatywnie wpłynąć na jakość produktu.

AT-4.1.5 (K3) Zastosować podział historyjki użytkownika w celu uzyskania testowalnych historyjek użytkownika.

4.2 Przesunięcie w lewo i inżynieria wymagań

AT-4.2.1 (K2) Wyjaśnić, jak inżynieria wymagań wspiera przesunięcie w lewo.

Wstęp

Przesunięcie w lewo kładzie nacisk na jak najwcześniejsze włączenie działań związanych z zapewnianiem jakości do cyklu życia oprogramowania, koncentrując się na poprawie podstawy testów od samego początku. Testalia takie jak kryteria akceptacji, listy kontrolne czy skrypty testów automatycznych, mogą pełnić rolę „żyjących wymagań” lub wspierać i dokumentować wymagania, zapewniając ich jasność i wspólne zrozumienie. Techniki takie jak *storyboarding* i *testboarding* pomagają wizualizować przepływy pracy i ujednolicić perspektywę, podczas gdy mapowanie przykładów udoskonala kryteria akceptacji poprzez wspólną dyskusję. Świadomość błędów poznawczych chroni przed błędnymi założeniami, które mogą obniżyć jakość produktu. Praktyki takie jak podział historyjek użytkownika na mniejsze części zapewniają, że historyjki użytkownika są krótkie, konkretne i testowalne. Równolegle skuteczna inżynieria wymagań wspiera przesunięcie w lewo poprzez połączenie analizy, dokumentacji i walidacji oraz poprzez wybór odpowiednich metod pozyskiwania wymagań, takich jak wywiady, warsztaty lub prototypowanie, w celu uchwycenia potrzeb interesariuszy. Dzięki przesunięciu w lewo zespół może szybciej wykrywać anomalie i defekty, ograniczać konieczność ponownej pracy oraz dostarczać produkty o wyższej wartości. Pojawiające się narzędzia oparte na sztucznej inteligencji i dużych modelach językowych mogą wspierać przesunięcie w lewo poprzez analizę wymagań pod kątem niejasności, sugerowanie scenariuszy testowych na podstawie kryteriów akceptacji lub identyfikowanie luk w pokryciu historyjek użytkownika. Testerzy powinni krytycznie oceniać takie narzędzia, rozumiejąc ich ograniczenia, i wykorzystywać je do uzupełnienia, a nie zastąpienia analizy przeprowadzanej wspólnie przez ludzi.

4.1. Użycie przesunięcia w lewo w celu zwiększenia jakości podstawy testów

4.1.1. Testalia jako wymagania

W zwinnym wytwarzaniu oprogramowania testalia takie jak historyjki użytkownika z kryteriami akceptacji, listy kontrolne, scenariusze BDD, akceptacyjne przypadki testowe, przykłady, karty opisu testu, kontrakty lub skrypty testów automatycznych, mogą być traktowane jako forma wymagań, gdy są wykorzystywane wspólnie oraz weryfikowane i stosowane w każdej iteracji. Te produkty pracy pomagają zespołom wyjaśnić oczekiwania, budować wspólne zrozumienie i wspierać bieżącą kontrolę zachowania oprogramowania. Często są one tworzone podczas rozmów z interesariuszami biznesowymi, co czyni je żyjącymi wymaganiami, a nie statyczną dokumentacją.

Wymagania stosowane w wytwarzaniu sterowanym zachowaniem (BDD), wytwarzaniu sterowanym testami akceptacyjnymi (ATDD) lub specyfikacji przez przykłady (Adzic, 2011) są wyprowadzane z rozmów między zespołem a interesariuszami biznesowymi i są często dokumentowane w formacie, który można wykonać jako testy. Te wykonywalne specyfikacje ucieleśniają kryteria akceptacji, służąc zarówno jako żyjące wymagania, jak i trwałe testy regresji.

W testowaniu eksploracyjnym opartym na sesjach karty opisu testu często wywodzą się z ogólnych pomysłów dotyczących produktu lub ocen ryzyka. Karty te kierują badaniem i pomagają zidentyfikować brakujące lub niezrozumiałe funkcje. Arkusze sesji testowych generowane podczas sesji testowania eksploracyjnego dostarczają nieformalnych, ale cennych spostrzeżeń na temat ewoluujących wymagań.

W podejściu DevOps i w ciągłym dostarczaniu logi z monitorowania, analityka zachowań klientów i pętle informacji zwrotnej mogą służyć jako ewoluujące wymagania, gdy zostaną zinterpretowane i przełożone na testowalne hipotezy lub reguły decyzyjne, zwłaszcza gdy zmiany są napędzane

przez zachowanie produkcji i dane dotyczące interakcji użytkowników. Zespoły mogą uchwycić te spostrzeżenia w przypadkach testowych lub listach kontrolnych w celu bieżącej walidacji.

Inne formy testaliów, takie jak mapy myśli, listy kontrolne i modele wizualne (np. diagramy przepływu lub diagramy stanów) mogą być wykorzystywane podczas zwinnego planowania i mogą bezpośrednio kierować pracami programistycznymi. Przekładają one domyślne zrozumienie na wspólne punkty odniesienia.

4.1.2. Storyboarding i testboarding

Storyboarding i *testboarding* to techniki oparte na współpracy, które poprawiają jakość podstawy testów poprzez wzmocnienie wspólnego zrozumienia oraz wczesne wykrywanie niejasności, ryzyk i pominięć. Z perspektywy testowania oprogramowania są one szczególnie skuteczne w zespołach zwinnych skupionych na informacji zwrotnej i współpracy całego zespołu.

Storyboarding wizualizuje ścieżki użytkownika, przedstawiając oczekiwane zachowanie jako sekwencję kroków lub scen (Walker i in., 2013). To narracyjne podejście ujawnia założenia, przypadki skrajne i interakcje, które mogą nie być oczywiste w wymaganiach tekstowych. Skłania do zadawania krytycznych pytań dotyczących logiki przepływu pracy, umożliwiając interesariuszom wykrycie niespójności lub niekompletnych przepływów przed napisaniem kodu. Ponieważ *storyboards* koncentrują się na interakcjach użytkownika, pomagają one odkryć luki w podstawie testów związane z użytecznością, przejściami między przepływami oraz potrzebami konkretnych person.

Testboarding to prosta praktyka, która pomaga wizualizować, co należy przetestować dla każdej funkcji, uzupełniając *storyboarding*. Oprócz weryfikacji kryteriów akceptacji, *testboarding* pokazuje główne i alternatywne scenariusze, co prowadzi do wczesnego uzgodnienia między testerami, programistami i przedstawicielami jednostek biznesowych w zakresie terminologii, zachowania systemu i priorytetów testowych. To wspólne zrozumienie zmniejsza ryzyko błędów interpretacyjnych podczas projektowania i wykonywania testów, zwiększając tym samym łatwość utrzymania przypadków testowych.

Łącząc te dwie techniki, zespoły tworzą wspólną przestrzeń do eksploracyjnego myślenia. Testerzy mogą zadawać pytania typu „co by było, gdyby?” i proponować scenariusze negatywne lub alternatywne, co skłania do wczesnego udoskonalenia historii użytkownika i jej kryteriów akceptacji. Praktyki, które pomagają wcześniej zbadać podstawę testów, poprawiają identyfikację ryzyka i ograniczają późne wykrywanie krytycznych defektów, umożliwiając testerom właściwe ustalanie priorytetów testów.

Techniki te są najskuteczniejsze, gdy stosuje się je iteracyjnie – ponowne analizowanie *storyboards* i *testboards* podczas dopracowywania backlogu i planowania iteracji pozwala dostrzec zmiany w zakresie, ryzykach lub zrozumieniu. Mogą one służyć jako żyjąca dokumentacja, która ewoluuje wraz z produktem i podstawą testów, pomagając testerom wykrywać sytuacje, w których wymagania ulegają zmianie lub zagrożone są cele jakościowe.

4.1.3. Mapowanie przykładów

Mapowanie przykładów (ang. *example mapping*) (Wynne, 2015) to technika oparta na współpracy, która służy do doprecyzowania historii użytkownika poprzez ustrukturyzowaną rozmowę (ang. *structured conversation*). Pomaga zespołom identyfikować reguły biznesowe, powiązane przykłady (które mogą stać się przypadkami testowymi) oraz zebrać pytania w celu pogłębienia wspólnego zrozumienia. Zespół tworzy kolorowe karty reprezentujące różne elementy dyskusji (np. żółte dla omawianej historii użytkownika, niebieskie dla reguł biznesowych, zielone dla przykładów ilustrujących reguły i czerwone dla pytań). Reguły są pogrupowane pod historijką użytkownika, a przykłady są pogrupowane pod odpowiednimi regułami. Zespół kontynuuje

tworzenie kart, aż uzna, że zakres historyjki użytkownika jest jasny lub upłynie wyznaczony czas. Gdy skończy się czas, może to oznaczać, że historyjka użytkownika jest zbyt złożona lub wymaga dopracowania, a zespół podejmuje decyzję co do kolejnych kroków.

W miarę rozwoju rozmowy zespół szybko tworzy wizualną reprezentację historyjki użytkownika, która odzwierciedla jej aktualne wspólne rozumienie. Zbyt wiele kart z pytaniami sugeruje, że zespół wciąż musi się wiele nauczyć o tej historyjce użytkownika. Duża liczba kart z regułami lub pojedyncza reguła z wieloma kartami z przykładami sugeruje, że historyjka użytkownika jest obszerna i złożona i powinna zostać podzielona (patrz sekcja 4.1.5). W takich przypadkach zespół rozdziela reguły i przykłady między pierwotną historyjkę a nowy fragment, tworzy kolejną żółtą kartę (historyjka użytkownika) dla podzielonej historyjki użytkownika i umieszcza ją w backlogu.

Z punktu widzenia testowania mapowanie przykładów wzmacnia podstawę testów poprzez osadzenie abstrakcyjnych wymagań w konkretnych, uzgodnionych przykładach. Przykłady te zmniejszają niejasności, odkrywają ukryte założenia i wcześniej ujawniają sprzeczne interpretacje.

W zwinnym wytwarzaniu oprogramowania podstawa testów często ewoluje podczas dopracowywania i wdrażania. Mapowanie przykładów poprawia jej jakość, zapewniając wspólne zrozumienie w całym zespole oraz wyjaśniając kryteria akceptacji, które bezpośrednio wpływają na wytwarzanie sterowane testami akceptacyjnymi (ATDD) i wytwarzanie sterowane zachowaniem (BDD). Promuje ono walidację w przesunięciu w lewo, ujawniając luki w specyfikacji przed rozpoczęciem kodowania.

Mapowanie przykładów wspiera tworzenie kart opisu testu do testowania eksploracyjnego i pomaga testerom zidentyfikować odpowiednie warunki testowe oraz alternatywne scenariusze, ponieważ konkretne przykłady i pytania w naturalny sposób sugerują testowalne granice i skrajne przypadki do zbadania. Stosowane iteracyjnie sprawia, że podstawa testów jest bardziej odporna na zmiany, ponieważ śledzenie powiązań od przykładów do reguł i potrzeb użytkowników jest zachowane przez cały rozwój.

4.1.4. Stronniczość w testowaniu zwinnym

Stronniczość (ang. *bias*) to systematyczne odchylenie od neutralności lub dokładności, które wpływa na to, jak ludzie myślą, oceniają lub działają, czasami odciągając ich od obiektywności lub sprawiedliwości. Stronniczości mogą znacznie obniżyć jakość produktu poprzez zniekształcanie postrzegania ryzyka, ograniczanie różnorodności testowania oraz osłabianie mechanizmów informacji zwrotnej, które są kluczowe dla praktyk zwinnego wytwarzania oprogramowania. W testowaniu stronniczości mogą wynikać z postaw jednostek, zespołów lub kultury organizacyjnej i nieświadomie wpływać na decyzje.

Stronniczości, które są szczególnie istotne w testowaniu zwinnym, obejmują:

- **Efekt potwierdzenia** (lub: błąd konfirmacji) (ang. *confirmation bias*) może prowadzić do projektowania i wykonywania testów, które weryfikują jedynie oczekiwane zachowanie, pomijając potencjalne ścieżki awarii lub nietypowe warunki. Wąski zakres eksploracji zwiększa ryzyko niewykrytych defektów, zwłaszcza w złożonych systemach, gdzie decyzje dotyczące rozwoju opierają się na ukrytych założeniach.
- **Efekt zakotwiczenia** (ang. *anchoring effect*) występuje, gdy wczesne wymagania lub oczekiwania interesariuszy powodują przywiązanie zespołów do konkretnych interpretacji, pomijając alternatywne zachowania. Na przykład, jeśli początkowe historyjki użytkownika koncentrują się wyłącznie na scenariuszach głównych (ang. *happy paths*), testowanie może nie uwzględniać w wystarczającym stopniu przypadków skrajnych, kwestii dostępności lub internacjonalizacji.

- **Błąd zgodności** (ang. *conformity bias*) może obniżyć jakość, gdy zniechęca się do różnorodnych opinii, gdy testerzy naśladują zachowania lub przekonania innych zamiast kierować się własnym, niezależnym osądem, lub gdy testerzy podporządkowują się dominującym głosom w zespołach wielofunkcyjnych. Podważa to wartość podejścia „cały zespół”, które opiera się na wspólnej odpowiedzialności za jakość i krytycznych pętlach informacji zwrotnej w całym cyklu życia oprogramowania.

Stronniczości mogą wpływać na projektowanie kryteriów akceptacji, zmniejszając przejrzystość i zwiększając niejasność w automatycznych testach akceptacyjnych lub wykonywalnych specyfikacjach. W praktykach takich jak specyfikacja przez przykłady i ATDD brak kwestionowania założeń podczas wspólnego tworzenia specyfikacji może prowadzić do nieefektywnego pokrycia.

Ryzyko wystąpienia stronniczości wzrasta w podejściu „cały zespół”. Aby je złagodzić, zespoły powinny promować różnorodność poglądów, wykorzystywać testowanie eksploracyjne w połączeniu z testowaniem w sesjach oraz pielęgnować kulturę bezpieczeństwa psychicznego, która zachęca do wyrażania sprzeciwu i zadawania pytań. Praktyki takie jak podsumowania (ang. *debriefs*) po sesjach testowania eksploracyjnego, testowanie w parach, wykorzystanie testerów jednorazowych oraz analiza podstawowej przyczyny mogą pomóc w ujawnieniu wpływu stronniczości i wzmocnieniu zbiorowej świadomości.

4.1.5. Podział historyjki użytkownika

Podział historyjki użytkownika (ang. *user story slicing*) to praktyka polegająca na rozbiciu dużych historyjek użytkownika na mniejsze, łatwiejsze do zarządzania części (ang. *slices*), które nadal dostarczają wartość, są wykonalne w ramach iteracji i testowalne. Prawidłowo podzielone historyjki użytkownika ułatwiają ciągłe dostarczanie i umożliwiają wcześniejsze uzyskanie informacji zwrotnej, wspierając w ten sposób zasady zwinnego wytwarzania oprogramowania.

Aby zastosować podział historyjki użytkownika w sposób wspierający testowalność, zespół zaczyna od zidentyfikowania różnych wymiarów, wzdłuż których można podzielić historyjkę użytkownika:

- **Podział według przepływu pracy** rozkłada historyjkę użytkownika w oparciu o zadania użytkownika lub etapy procesu biznesowego. Każda część powinna skupiać się na jednym zadaniu lub interakcji, najlepiej takiej, która skutkuje obserwowalnym zachowaniem systemu. Ułatwia to zdefiniowanie precyzyjnych kryteriów akceptacji i oczekiwanych wyników.
- **Podział według złożoności danych** zaczyna się od jednego przykładu scenariusza głównego (ang. *happy path*) z wykorzystaniem najprostszych danych, a alternatywne warunki danych i przypadki skrajne przenosi do oddzielnych historyjek użytkownika. Pozwala to zespołowi na wczesną weryfikację podstawowej funkcjonalności i stopniowe zwiększanie pokrycia. Każda część powinna zawierać tylko tyle wariacji, aby umożliwić przeprowadzenie znaczących testów bez nadmiernego zwiększenia zakresu.
- **Podział według interfejsu** oddziela logikę przetwarzania backendu od funkcjonalności interfejsu użytkownika, gdy każda z nich może być testowana niezależnie. Umożliwia to wcześniejszą automatyzację testów i jest zgodne z warstwowością architektury, zwłaszcza w środowiskach ciągłej integracji. Zespół tworzy historyjki użytkownika dotyczące backendu, które udostępniają wyniki za pośrednictwem interfejsów API, zanim rozpocznie pracę nad historyjkami użytkownika obejmującymi przepływy pracy w interfejsie użytkownika.
- **Podział według scenariuszy** zaczyna się od najczęstszego scenariusza biznesowego i opiera się na nim, tworząc dodatkowe części dla rzadszych lub bardziej złożonych przypadków. Zespół może wykorzystać mapowanie przykładów lub ustrukturyzowane

rozmowy, aby wydobyć te scenariusze podczas dopracowywania. Każda część scenariusza powinna być weryfikowalna poprzez konkretne przykłady, które stanowią podstawę dla testów zorientowanych na biznes.

- **Podział pionowy** gwarantuje, że każda historyjka użytkownika zawiera kompletną, wąską część funkcjonalności we wszystkich warstwach, od bazy danych po interfejs użytkownika, tam, gdzie to stosowne. Pomaga to zespołom dostarczać wymierną wartość, przeprowadzać testy kompleksowe (ang. *end-to-end*) oraz weryfikować integrację między modułami. Należy jednak unikać umieszczania zbyt wielu elementów w jednej części. Jeśli zakres testu kompleksowego stanie się niemożliwy do zarządzania, zespół powinien powrócić do podziału według scenariuszy lub przepływu pracy.
- **Podział oparty na ograniczeniach** koncentruje się najpierw na ogólnych założeniach, a następnie uwzględnia warunki wyprowadzone ze znanych ograniczeń. Na przykład zespół zaczyna od historyjki użytkownika, która zakłada, że użytkownik jest uwierzytelniony, a następnie wyodrębnia oddzielną historyjkę użytkownika, aby zająć się awariami uwierzytelniania. Pomaga to wyodrębnić warunki testowe i zwiększa przejrzystość podczas definiowania granic.

Kryteria akceptacji często decydują o powodzeniu lub niepowodzeniu podziału historyjki użytkownika. Podczas podziału zespół nie może po prostu „rozłożyć” historyjki użytkownika na kawałki; musi przemyśleć i przedyskutować kryteria akceptacji, tak aby każda część miała sens. Na przykład duże historyjki użytkownika zazwyczaj zawierają zestaw ogólnych kryteriów akceptacji, które po podziale należy doprecyzować, aby odpowiadały węższemu zakresowi. Kryteria akceptacji powinny skupiać się na jednym wąskim wyniku dla użytkownika na każdą część; należy unikać łączenia wielu wariantów zachowań w jednej części. Podczas dzielenia na części nie należy kopiować i wklejać oryginalnych kryteriów akceptacji do każdej z nich. Zamiast tego należy je przepisać dla każdej części, tak aby każde z nich było testowalne, można je było zademonstrować i by odpowiadało danej części. Zestaw kryteriów akceptacji dla wszystkich części powinien ostatecznie obejmować całą oryginalną historyjkę użytkownika.

Niezależnie od strategii podziału, zespół powinien zawsze traktować testowalność jako kryterium nadrzędne. Odbyna się to poprzez zadanie pytania, czy część można zweryfikować niezależnie, czy można wyprowadzić jasne kryteria akceptacji oraz czy wspiera ona tworzenie testów zorientowanych na biznes. Jeśli którykolwiek z tych warunków nie jest spełniony, historyjka użytkownika wymaga dalszego dopracowania.

4.2. Przesunięcie w lewo i inżynieria wymagań

Przesunięcie w lewo przenosi działania związane z jakością na wcześniejszy etap cyklu życia oprogramowania, aby wcześniej wykrywać anomalie i defekty, ograniczyć przeróbki, skrócić cykle dostaw i poprawić dostosowanie do potrzeb biznesowych. Wszystkie główne działania w inżynierii wymagań – pozyskiwanie, analiza, specyfikacja i walidacja wymagań – mogą wspierać przesunięcie w lewo poprzez ograniczanie nieporozumień, zapobieganie defektom i minimalizowanie kosztownych przeróbek w późniejszych etapach cyklu życia oprogramowania:

- Pozyskiwanie wymagań wspiera przesunięcie w lewo poprzez wczesne i ciągłe rejestrowanie potrzeb interesariuszy, umożliwiając rozpoczęcie prac programistycznych i testowych przy jasnym zrozumieniu oczekiwań interesariuszy. Ciągłe pozyskiwanie wymagań poprzez historyjki użytkownika, porządkowanie backlogu i warsztaty oparte na współpracy zapewnia, że funkcje są zdefiniowane, testowalne i zweryfikowane, co ogranicza liczbę defektów i konieczność ponownej pracy w późniejszych fazach iteracji.

- Analiza wymagań wspiera przesunięcie w lewo poprzez wczesne identyfikowanie niejasności, konfliktów, ryzyk i zależności, zapewniając, że prace programistyczne i testowe rozpoczynają się przy jasnych, wykonalnych i uszeregowanych pod względem priorytetów wymaganiach. Analizując wymagania, zespoły mogą projektować odpowiednie rozwiązania, planować testy i zapobiegać defektom przed rozpoczęciem kodowania, co ogranicza kosztowne poprawki w późniejszych fazach cyklu życia oprogramowania.
- Specyfikacja wymagań wspiera przesunięcie w lewo poprzez zapewnienie jasnej, precyzyjnej i testowalnej dokumentacji tego, co system musi robić, umożliwiając programistom i testerom wczesne planowanie i walidację pracy. Dobrze określone wymagania ograniczają nieporozumienia, pomagają we wczesnym tworzeniu testów i pozwalają wykrywać anomalie przed wdrożeniem, minimalizując defekty i przeróbki na dalszych etapach. Testerzy stosujący metodyki zwinne zazwyczaj pracują ze swobodnymi formami tekstowymi, takimi jak historyjki użytkownika i opowieści. Gdy potrzebna jest większa struktura, mogą oni korzystać z półformalnych modeli scenariuszy, takich jak przypadki użycia, modele procesów biznesowych (np. w notacji BPMN) lub diagramy przejść pomiędzy stanami, w zależności od kontekstu. Modele scenariuszy mogą ilustrować przepływy interakcji, w tym przypadki skrajne, które mogą nie zostać uchwycone w opisach wysokiego poziomu. Często utrzymywane są słowniki, aby zapewnić wspólne rozumienie terminów specyficznych dla danej dziedziny, zwłaszcza podczas definiowania kryteriów akceptacji.
- Walidacja wymagań wspiera przesunięcie w lewo poprzez zapewnienie, że wymagania są poprawne, jednoznaczne, kompletne, spójne i możliwe do przetestowania przed rozpoczęciem rozwoju. Wczesna walidacja poprzez przeglądy, prototypowanie i modelowanie pomaga wykrywać anomalie i luki, zmniejszając liczbę defektów, przeróbek i kosztownych zmian w późnych fazach.

5. Zwinne podejścia do testów i techniki testowania (285 min.)

Słowa kluczowe

heurystyka testowa, karta opisu testu, mnemonik testowy, przypadek skrajny (ang. *edge case*), przypadek testowy, testowanie eksploracyjne, testowanie grupowe (ang. *mob testing*), testowanie w parach (ang. *pair testing*), testowanie intuicyjne (ang. *vibe testing*), wędrówka testowa (ang. *tour*), zapach testu (ang. *test smell*), zwinne wytwarzanie oprogramowania

Słowa kluczowe specyficzne dla dziedziny

FEW HICCUPPS, I SLICED UP FUN, opowieść (ang. *epic*), RCRCRC, SFDIPOT, TERMS

Cele nauczania dla rozdziału 5

5.1 Testowanie eksploracyjne

AT-5.1.1 (K2) Wyjaśnić heurystyki testowe.

AT-5.1.2 (K2) Podać przykłady mnemoników testowych związanych z testowaniem w zwinnym wytwarzaniu oprogramowania.

AT-5.1.3 (K2) Wyjaśnić wędrówki testowe.

AT-5.1.4 (K4) Przeanalizować historyjki użytkownika i opowieści w celu utworzenia kart opisu testu.

AT-5.1.5 (K3) Zastosować testowanie eksploracyjne w celu wsparcia testowania w zwinnym wytwarzaniu oprogramowania.

5.2 Testowanie wspomagane

AT-5.2.1 (K2) Wyjaśnić testowanie grupowe.

AT-5.2.2 (K2) Wyjaśnić testowanie w parach.

AT-5.2.3 (K2) Wyjaśnić testowanie intuicyjne.

5.3 Zapachy testów

AT-5.3.1 (K3) Użyć zapachów testów do oceny jakości przypadków testowych.

Wstęp

Testerzy muszą dostosować swoje strategie testowe i narzędzia do szybkiego tempa i iteracyjnego charakteru projektów zwinnego wytwarzania oprogramowania. W niniejszym rozdziale omówiono zestaw technik testowania i metod współpracy, które zwiększają skuteczność i wydajność testowania w ramach tego podejścia. Kładą one nacisk na elastyczność, informacje zwrotne w czasie rzeczywistym oraz ciągłe uczenie się – podstawowe wartości metodyk zwinnego wytwarzania oprogramowania.

5.1. Testowanie eksploracyjne

5.1.1. Heurystyki testowe

Heurystyki testowe to ogólne wytyczne lub praktyczne zasady, które pomagają testerom podejmować świadome decyzje, rozwiązywać problemy i kierować badaniem testowanego systemu. Nie ma gwarancji, że będą one poprawne w każdym kontekście, ale służą jako praktyczne strategie, które często prowadzą do wartościowych wyników, gdy:

- czas lub zasoby są ograniczone,
- specyfikacja jest niekompletna lub niejasna,
- tester przeprowadza testy oparte na doświadczeniu,
- testy regresji są przeprowadzane bez pełnej automatyzacji testów,
- tester chce myśleć jak użytkownik lub symulować rzeczywiste przypadki użycia.

Przykłady heurystyk testowych obejmują:

Wytyczne (ang. *guidelines*) – zalecane procedury lub kroki, które wskazują testerom konkretne działania, jakie należy podjąć, lub podejścia, których należy przestrzegać podczas testowania. Przykłady obejmują: „testuj wcześniej i często”, „testowanie dotyczy informacji, a nie tylko wyniku pozytywnego/negatywnego”, „zawsze bierz pod uwagę granice”, „wypróbuj typowe przepływy pracy oraz nieoczekiwane lub nieprawidłowe zachowania”, „zmieniaj po jednej zmiennej na raz, aby wyodrębnić efekty podczas testowania opcji, konfiguracji lub danych wejściowych”, „testuj, jak funkcje współpracują ze sobą w sekwencji, a nie tylko w izolacji”.

Ogólne listy kontrolne (ang. *generic checklists*) – listy typowych elementów do przetestowania, które można wykorzystywać wielokrotnie, służące jako przypomnienia lub wskazówki do zbadania obszarów, które w przeciwnym razie mogłyby zostać pominięte. Listy kontrolne mogą skupiać się na różnych obszarach, takich jak charakterystyki jakościowe, typy testów, poziomy testów, obszary ryzyka, role użytkowników i środowiska testowe. Przykłady obejmują heurystyki użyteczności wg Nielsena (Nielsen, 1994), 10 najpoważniejszych ryzyk zabezpieczeń aplikacji internetowych według OWASP (OWASP, 2025) czy taksonomie defektów, takie jak taksonomia Beizera (Beizer, 1990) oraz taksonomia wg normy IEEE 1044 (IEEE 1044, 2009).

Praktyczne zasady (ang. *rules of thumb*) – niepisane „prawa” wynikające z doświadczenia lub powszechnie akceptowane prawdy. Przykłady obejmują: „jeśli coś wygląda na zbyt proste, prawdopodobnie kryje w sobie złożoność”, „jeśli coś się zepsuło wcześniej, prawdopodobnie zepsuje się ponownie”, „defekty mogą się kumulować”, „jeśli coś jest trudne do przetestowania, prawdopodobnie jest trudne w użyciu”, „to, co działa u jednego użytkownika, może zawieść u wielu”.

Mnemoniki – pomoce pamięciowe, które pomagają testerom szybko przywołać zestaw warunków testowych. Przykładem jest SFDIPOT (patrz sekcja 5.1.2).

Analogie i metafory – pozwalają testerom porównać system do znanych pojęć, a następnie zastosować rozumowanie oparte na tych pojęciach w celu wygenerowania warunków testowych. Przykłady obejmują wędrówki testowe (ang. *test tours*) (patrz sekcja 5.1.3).

5.1.2. Mnemoniki testowe

Mnemoniki to pomoce pamięciowe, zazwyczaj w formie akronimów, skrótów, rymów lub wzorów. Mnemoniki nie narzucają sztywno, co należy testować, ale pomagają testerom przywołać złożone zestawy informacji i poszerzyć perspektywę podczas testowania eksploracyjnego i przeglądów. Jednak, aby były skuteczne, wymagają wiedzy o dziedzinie, kontekście i ryzyku, a także krytycznego myślenia i zaufania w zespole. W testowaniu oprogramowania mnemoniki testowe są powszechnie używane do:

- kierowania testowaniem eksploracyjnym poprzez nadanie struktury testom o swobodnej formie, bez sztywnych przypadków testowych,
- pobudzanie kreatywnego i krytycznego myślenia poprzez zachęcanie testerów do myślenia w sposób, którego naturalnie by nie rozważali,
- osiągnięcie szerszego zasięgu poprzez zapewnienie, że kluczowe obszary nie zostaną pominięte,
- służenia jako mentalne listy kontrolne podczas planowania lub wykonywania testów, zmniejszając obciążenie poznawcze poprzez skondensowanie złożonych pojęć w łatwe do zapamiętania formaty.

W testowaniu oprogramowania stosuje się różne mnemoniki testowe, testerzy mogą też tworzyć własne ich zestawy. Niektóre popularne mnemoniki to:

- SFDIPOT (*Structure, Function, Data, Interfaces, Platform, Operations, Time*) – wspiera ogólną analizę testowanego systemu (Bolton, 2014). Można go używać podczas eksploracji nowego systemu lub planowania kart opisu testu.
- RCRCRC (*Recent, Core, Risky, Configuration, Repaired, Chronic*) – wspiera testy regresji, gdzie każda litera reprezentuje słowo pomagające w odkrywaniu warunków testowych (Johnson, 2012). Można go używać do ustalania pokrycia w iteracjach zwinnego wytwarzania oprogramowania lub cyklach testów regresji.
- I SLICED UP FUN (*Inputs, Store, Location, Interactions/Interruptions, Communication, Ergonomics, Data, Usability, Platform, Function, User scenarios, Network*) (Kohl, 2010). Można go wykorzystać do wspierania testowania aplikacji mobilnych przy generowaniu warunków testowych.
- FEW HICCUPPS (*Familiarity, Explainability, World, History, Image, Comparable products, Claims, Users' desires, Product, Purpose, Statutes*) – wspiera identyfikację i zastosowanie wyroczeni testowych (Bolton, 2012). Można go wykorzystać do identyfikacji defektów poprzez obserwację różnych rodzajów niespójności.
- TERMS (*Tools and Technology, Execution, Requirements and Risks, Maintenance, Security*) (Gareev, 2019). Pomaga zapamiętać różne czynniki, które wpływają na sukces automatyzacji testów.

5.1.3. Wędrówki testowe

Wędrówki testowe (ang. *test tours*) wykorzystują metaforę turystycznych wędrówek po mieście, aby poprowadzić testowanie eksploracyjne systemu. Koncepcja ta ma na celu pomóc testerom w eksploracji oprogramowania, podobnie jak turyści eksplorują nowe miejsce, wykorzystując różne rodzaje „wędrówek” w celu odkrycia rozmaitych problemów. Każda wędrówka koncentruje się na konkretnej perspektywie, prowadząc testera do zbadania aplikacji z określonego punktu widzenia. Wędrówki są zaprojektowane tak, aby były nieformalne, lekkie i kreatywne, co czyni je

idealnymi do testowania eksploracyjnego, testowania w sesjach lub jako generatory warunków testowych.

Przykłady wędrówek testowych, inspirowanych typami dzielnic miejskich (Whittaker, 2009), podane są poniżej.

Dzielnica biznesowa. W przypadku oprogramowania odnosi się to do podstawowych funkcji. Obszar ten jest ograniczony kodem uruchamiającym i wyłączającym i zawiera cechy i funkcje, z którymi użytkownicy wchodzi w interakcję w oprogramowaniu.

Dzielnica historyczna. W przypadku oprogramowania odnosi się to do kodu zastanego (ang. *legacy code*) oraz historii wadliwych funkcji lub cech. Podobnie jak w przypadku prawdziwej historii, kod zastany jest często słabo rozumiany, a podczas jego włączania, modyfikowania lub używania przyjmuje się wiele założeń.

Dzielnica turystyczna. Wiele miast posiada dzielnice, które odwiedzają głównie turyści. Miejscowi zazwyczaj unikają tych obszarów. Oprogramowanie odzwierciedla to zjawisko, ponieważ początkujący użytkownicy skłaniają się ku cechom i funkcjom, których doświadczeni użytkownicy już nie potrzebują.

Dzielnica rozrywkowa. W miastach dzielnice te oferują relaks i rozrywkę. Oprogramowanie, podobnie jak miasta, również posiada funkcje wspierające. Te wędrówki uzupełniają wędrówki przez inne dzielnice i wypełniają luki w kompleksowym planie testów.

Dzielnica hotelowa. W miastach dzielnice te są miejscem odpoczynku. W kontekście testowania jest to miejsce, w którym tester oprogramowania może odejść od podstawowej funkcjonalności i popularnych funkcji, aby przetestować niektóre funkcje drugorzędne i pomocnicze, które często są pomijane lub niedostatecznie uwzględnione w planie testów.

Dzielnice o złej reputacji są niebezpieczne, ale nadal przyciągają pewną grupę turystów. Wędrówki po nich są niezbędne dla testerów, ponieważ pozwalają im odkryć obszary podatności, które mogą stanowić poważny problem dla użytkowników, jeśli pozostaną w produkcji. Obszar ten dotyczy przede wszystkim testowania negatywnego i wykorzystuje techniki testowania, takie jak zgadywanie błędów lub ataki usterek.

5.1.4. Tworzenie karty opisu testu

Karty opisu testu są ściśle powiązane z zasadami zwinnego wytwarzania oprogramowania i zapewniają ustrukturyzowane, a jednocześnie elastyczne podejście do testowania eksploracyjnego (patrz sekcja 5.1.5). Karta opisu testu to krótki, konkretny dokument, który określa cel, zakres i cele testów sesji testowania eksploracyjnego. Reprezentuje ona przewodnią misję lub testowalną hipotezę dla testera podczas sesji testowej. Karty opisu testu mogą być tworzone w oparciu o różne źródła, takie jak historyjki użytkownika, kryteria akceptacji, ryzyka produktowe lub cele iteracji. Karta opisu testu zawiera co najmniej:

- misję/cele testów – ogólny kierunek lub testowalne cele sesji (np. „zweryfikuj logowanie użytkownika pod obciążeniem”),
- zakres – konkretne obszary zainteresowania w testowanym systemie, poziom testów, warunki testowe, techniki testowania, które mają być użyte, kryteria wyjścia, priorytety, co ma być pokryte testami, a co nie

i może również zawierać inne informacje, takie jak:

- aktor – docelowy użytkownik systemu,
- cel – cel, który aktor chce osiągnąć,

- konfiguracja – co musi być gotowe, aby rozpocząć testowanie eksploracyjne (np. środowisko testowe),
- priorytet – względne znaczenie niniejszej karty opisu testu, oparte na priorytecie powiązanej historyjki użytkownika lub poziomie ryzyka,
- odniesienie – specyfikacje (np. historyjka użytkownika), ryzyka lub inne źródła informacji,
- dane – wszelkie dane potrzebne do realizacji karty opisu testu,
- czynności – lista pomysłów dotyczących tego, co aktor może chcieć zrobić z systemem, co warto przetestować i jakie techniki testowania można zastosować,
- uwagi dotyczące wyroczni testowej – jak oceniać produkt w celu ustalenia prawidłowych wyników testów (np. w celu uchwycenia tego, co dzieje się na ekranie i porównania tego z tym, co jest napisane w instrukcji obsługi),
- wariacje – alternatywne działania i oceny uzupełniające pomysły opisane w punkcie „czynności”,
- ograniczenia – np. czego produkt nigdy nie może robić,
- uwarunkowania i ryzyka – np. stosowane przepisy, zasady i normy.

Aby opracować karty opisu testu na podstawie historyjek użytkownika i opowieści, testerzy zwinni analizują je w celu zidentyfikowania celów testów i głównej misji. Wydobywają intencję historyjki użytkownika lub opowieści (np. kim jest aktor i jaki jest jego cel) i przekładają ją na misję sesji testowej. Powiązane historyjki użytkownika można pogrupować w celu oceny zintegrowanego zachowania.

Testerzy zwinni określają, co należy poznać, zweryfikować lub zbadać podczas sesji testowej. Mogą oni wykorzystać technikę 5W1H, znaną również jako metoda Kiplinga, aby wyczerpująco odpowiedzieć na istniejące pytania i wygenerować pomysły, które mogą przyczynić się do rozwiązania problemu:

- Jaka funkcja jest dostarczana?
- Kim są użytkownicy?
- Kiedy funkcja jest używana?
- Gdzie funkcja jest używana?
- Dlaczego ta funkcja jest ważna?
- Jak funkcja integruje się z innymi funkcjami?

Te pytania pomagają określić zakres, konfigurację i priorytet. Różne heurystyki testowe (patrz sekcje 5.1.1, 5.1.2 i 5.1.3) mogą być pomocne w udzieleniu odpowiedzi na te pytania oraz w generowaniu warunków testowych, danych testowych i czynności testowych, które należy uwzględnić w karcie opisu testu.

Kryteria akceptacji określają, co musi być spełnione, aby historyjka użytkownika została zaakceptowana. Można je bezpośrednio przełożyć na warunki testowe. Chociaż kryteria akceptacji wyznaczają oczekiwane zachowanie, wskazują one również obszary do zbadania poza scenariuszem głównym: co się stanie, jeśli kryteria akceptacji nie zostaną spełnione? Jakie pojawiają się przypadki skrajne lub negatywne warunki testowe? Czy kryteria akceptacji są jasne, czy też mogą być niejednoznaczne? Pomaga to testerom w opracowaniu warunków testowych zarówno dla testów pozytywnych, jak i negatywnych. Kryteria akceptacji często sugerują lub określają wymagania dotyczące danych, takie jak prawidłowe wartości wejściowe, warunki wstępne lub konkretne formaty lub zakresy.

Kryteria akceptacji mogą również służyć jako uproszczone notatki wyroczni testowej, ponieważ definiują oczekiwane zachowania, ustalają progi zaliczenia/niezaliczenia i są zgodne z oczekiwaniami przedstawicieli jednostek biznesowych. Testerzy powinni jednak pamiętać, że kryteria akceptacji nie są wyczerpującymi wyroczniami testowymi – nie uwzględniają wszystkich

aspektów jakości. Testy eksploracyjne są nadal potrzebne do wykrywania defektów wykraczających poza zakres kryteriów akceptacji. Testerzy stosujący metodyki zwinne powinni traktować brakujące, niejasne lub niejednoznaczne kryteria akceptacji jako okazję do przeprowadzenia testów eksploracyjnych w celu wykrycia nieokreślonych wariantów i przypadków skrajnych.

5.1.5. Wykonywanie testowania eksploracyjnego

W środowiskach zwinnego wytwarzania oprogramowania, gdzie wymagania często się zmieniają, testowanie eksploracyjne oferuje elastyczność i szybkość reakcji, których często brakuje w podejściach opartych na testach skryptowych (ang. *scripted approaches*). Testowanie eksploracyjne jest również ważne w projektach zwinnego wytwarzania oprogramowania ze względu na ograniczony czas dostępny na analizę testów oraz ograniczoną szczegółowość historyjek użytkownika.

Testowanie eksploracyjne można stosować w różnych momentach cyklu życia zwinnego wytwarzania oprogramowania, w tym:

- podczas wykonywania iteracji (w celu szybkiego przetestowania nowych funkcji lub zmian przed lub równoległe z testowaniem skryptowym),
- podczas przeglądów/prezentacji wyników iteracji (w celu nieformalnego zbadania produktu przy jednoczesnym zbieraniu informacji zwrotnych),
- po wprowadzeniu poważnych zmian (w celu wykrycia defektów integracyjnych lub regresji, które mogą zostać pominięte w testach skryptowych),
- gdy kryteria akceptacji są niejasne lub szcątkowe (w celu wyjaśnienia oczekiwań i znalezienia przypadków skrajnych).

Testowanie eksploracyjne opiera się zazwyczaj na kartach opisu testu i jest przeprowadzane w ramach sesji testowych (Ghazi i in., 2017). Sesja testowa to ograniczony czasowo, nieprzerwany okres (zwykle 60–120 minut). Sesje te pomagają zorganizować działania testowe i zapewniają strukturę w ramach elastycznego podejścia do testów. Sesja testowa koncentruje się na konkretnej karcie opisu testu i często jest dokumentowana za pomocą arkuszy sesji testowej, obserwacji i ustaleń. Typowa struktura sesji testowania eksploracyjnego wygląda następująco:

- Przygotowanie przed sesją. Określa się kartę opisu testu, przygotowuje środowisko testowe i ustala ramy czasowe sesji testowej.
- Faza wykonywania testów. Tester zapoznaje się z działaniem produktu, bada go i ocenia. Tester postępuje zgodnie z założeniami karty opisu testu, ale dostosowuje się w miarę odkrywania nowych informacji. Wykorzystuje heurystyki i mnemoniki testowe do odtworzenia warunków testowych. Ponieważ testowanie eksploracyjne jest testowaniem swobodnym, odpowiednia dokumentacja jest niezbędna do odtworzenia wyników, sporządzenia raportów z testów i wprowadzenia udoskonaleń. Standardowe metody sporządzania notatek obejmują arkusze sesji testowych, notatki w dowolnej formie, zrzuty ekranu, nagrania i mapy myśli. Udokumentowane informacje zazwyczaj obejmują pokrycie (w tym pokrycie ryzyka), notatki z oceny, zapis rzeczywistego zachowania systemu oraz wykryte anomalie.
- Przegląd po sesji. Tester eksploracyjny omawia wyniki sesji testowej z interesariuszami, w szczególności:
 - porównanie karty opisu testu z rzeczywistością (czy tester postępował zgodnie z kartą opisu testu, czy znalazł nieoczekiwane obszary i osiągnął cele sesji?),
 - znalezione defekty,
 - zgłoszone pytania (np. niejasne wymagania lub niepewności techniczne),
 - kolejne kroki (czy potrzebne są nowe karty opisu testu lub testy regresji?).

Podczas wykonywania testów eksploracyjnych tester nie ma żadnych ograniczeń dotyczących podejść do testów, technik testowania ani typów testów. Może stosować czarnoskrzynkowe i białoskrzynkowe techniki testowania. Jednak ze względu na charakter testów eksploracyjnych najczęściej stosuje się testowanie oparte na doświadczeniu oraz heurystyki testowe (patrz sekcje 5.1.1, 5.1.2 i 5.1.3).

Testerzy eksploracyjni muszą stosować wyrocznie testowe w sposób elastyczny i kontekstowy, często opierając się na swoim doświadczeniu, wiedzy o produkcie i współpracy z zespołem. W testowaniu eksploracyjnym wyrocznie testowe:

- są często zgodne z historyjkami użytkowników lub kryteriami akceptacji (patrz sekcja 5.1.4),
- wyrażają zdrowy rozsądek lub oczekiwania użytkowników,
- są spójne z podobnymi funkcjami lub poprzednimi wersjami,
- są zgodne ze standardami lub wytycznymi (np. standardami dostępności lub interfejsu użytkownika).

Kategorie wyroczni testowych wspierających testowanie eksploracyjne mogą obejmować mnemonik testowy FEW HICCUPPS (patrz sekcja 5.1.2).

5.2. Testowanie wspomagane

5.2.1. Testowanie grupowe

Testowanie grupowe (znane również jako testowanie zespołowe, ang. *mob testing*, *ensemble testing*) to podejście oparte na współpracy, w ramach którego niewielka grupa (zazwyczaj 5–8 osób) testuje ten sam system jednocześnie. Opiera się ono na koncepcji „roju” (ang. *swarming*), w której wielu członków zespołu skupia się na danym problemie testowym. Ta współpraca w czasie rzeczywistym łączy różnorodne umiejętności, perspektywy i daje natychmiastową informację zwrotną w celu poprawy jakości, szybkości i kreatywności testowania.

Konfiguracja testowania grupowego obejmuje wspólny ekran, komputer i tablicę (w przypadku pracy zdalnej środowisko może być wirtualne, a nie fizyczne). Wykorzystuje następujące role:

- Moderator – koordynator sesji, który nadzoruje jej przebieg, dba o to, by wszyscy trzymali się swoich ról, zarządza czasem i dba o równomierny udział wszystkich uczestników.
- Nawigator (ang. *navigator*) – główny tester, który podejmuje ostateczne decyzje dotyczące działań, uwzględniając spostrzeżenia i wskazówki od zespołu.
- Operator (ang. *driver*) – osoba wprowadzająca w życie zbiorowe instrukcje zespołu, która zazwyczaj nie przejmuje inicjatywy w zakresie strategii, zapewniając, by pomysły najpierw przeszły przez umysł innej osoby.
- Grupa (ang. *mob*) – grupa obserwatorów, którzy monitorują postępy i w razie potrzeby dzielą się spostrzeżeniami.

Testowanie grupowe składa się z dwóch części: sesji grupowej i retrospektywy. Sesja grupowa rozpoczyna się od ustawienia osób w kręgu, z operatorem przy klawiaturze. Uczestnicy zmieniają się co jakiś czas (zwykle co 4 minuty); nawigator staje się operatorem, operator staje się pierwszą osobą w grupie i tak dalej.

Moderator siedzi z tyłu i nie zmienia się z resztą zespołu. Jeśli konieczna jest jego interwencja, może wstrzymać zespół i przyjąć dowolną rolę, z wyjątkiem roli operatora. Skład grupy zmienia się często, co zmusza wszystkich do skupienia uwagi. Sesja grupowa trwa zazwyczaj 2 godziny.

Po zakończeniu sesji następuje retrospektywa, mająca na celu udoskonalenie procesu na przyszłe sesje. Zazwyczaj moderator zbiera uwagi od każdego uczestnika na karteczkach

samoprzylepnych. Następnie zespół grupuje podobne uwagi, aby odkryć wzorce i możliwości ciągłego doskonalenia.

5.2.2. Testowanie w parach

Testowanie w parach to podejście do testów, w którym dwie osoby współpracują na jednym stanowisku pracy (lub w środowisku wirtualnym w przypadku pracy zdalnej lub zespołu rozproszonego) w celu przetestowania tego samego przedmiotu testów w czasie rzeczywistym (np. tester + tester, tester + programista, tester + analityk). Zazwyczaj podczas testowania w parach jedna osoba obsługuje komputer (operator), a druga obserwuje, sprawdza, robi notatki i opracowuje strategię (nawigator). Testowanie w parach czerpie inspirację z programowania w parach, praktyki spopularyzowanej przez programowanie ekstremalne (XP).

Testowanie w parach wspiera dzielenie się wiedzą, zachęcając do uczenia się od siebie nawzajem w zakresie wiedzy specjalistycznej i strategii testowych, i jest szczególnie przydatne podczas wdrażania nowych członków zespołu. Praca w parach pomaga szybciej wykrywać defekty dzięki synergii wynikającej ze współpracy. Poprawia to również pokrycie testowe poprzez połączenie różnych perspektyw. Informacja zwrotna w czasie rzeczywistym pomaga udoskonalić warunki testowe, co prowadzi do bardziej kreatywnych i skutecznych przypadków testowych. Może również poprawić komunikację poprzez ograniczenie nieporozumień dotyczących wymagań lub funkcjonalności. Testowanie w parach angażuje testerów i zmniejsza prawdopodobieństwo pominięcia kroków testowych. Natychmiastowa pętla informacji zwrotnej może nawet spowodować powstanie natychmiastowych pomysłów na usunięcie defektów (które nadal muszą być odpowiednio zgłaszane), zwłaszcza w parach programista-tester.

Testowanie w parach jest szczególnie cenne, gdy współpraca, nauka i szybka informacja zwrotna mają kluczowe znaczenie. W szczególności jest pomocne w:

- testowaniu eksploracyjnym,
- testowaniu złożonych lub nowych funkcji,
- testowaniu międzyfunkcyjnym (np. interfejs użytkownika + API),
- mentoringu młodszych testerów lub wzajemnych szkoleń (ang. *cross-training*) członków zespołu.

Testowanie w parach nie jest pozbawione wyzwań. Jest czasochłonne i wymaga dużego nakładu pracy, ponieważ dwie osoby pracują nad tym samym zadaniem, co może wydawać się mniej wydajne niż praca w pojedynkę. Ponadto testerzy mogą mieć różne style pracy, poziomy asertywności lub sposoby myślenia. Może to być zaletą, ale może również prowadzić do tarć, dyskomfortu i zmniejszenia wydajności. Jedna osoba może zdominować sesję (np. zawsze kierując pracą lub podejmując decyzje), a ciągłe mówienie, głośne myślenie i uzasadnianie decyzji może być wyczerpujące psychicznie.

Aby sprostać tym wyzwaniom:

- testowanie w parach powinno być stosowane strategicznie – np. tylko w przypadku funkcji złożonych lub wysokiego ryzyka,
- należy praktykować otwartą komunikację i aktywne słuchanie,
- należy jasno zdefiniować role (operator i nawigator) i regularnie je zmieniać, aby uniknąć utrwalonych wzorców zachowań,
- przed rozpoczęciem każdej sesji testowania w parach należy wyznaczyć jasne cele.

5.2.3. Testowanie intuicyjne

Testowanie intuicyjne (ang. *vibe testing*) to nowe, nieformalne podejście do testowania wspomaganego przez sztuczną inteligencję, które nie zostało jeszcze w pełni ustandaryzowane.

Rozwinęło się ono wraz z rozwojem kodu generowanego przez duże modele językowe (LLM), powszechnie nazywanym programowaniem intuicyjnym (ang. „*vibe coding*”). Zazwyczaj programiści manualnie piszą kod, tworzą przypadki testowe i postępują zgodnie z ustrukturyzowanymi procedurami kontroli jakości. Jednak w przypadku programowania intuicyjnego programiści opisują wymagane zachowanie aplikacji w języku naturalnym, zazwyczaj jako polecenia dla LLM, które następnie generują kod, często zmniejszając potrzebę szczegółowej weryfikacji przez człowieka. Ta zmiana w rozwoju wymaga podejścia testowego – zwanego testowaniem intuicyjnym – w celu sprawdzenia, czy produkt końcowy jest zgodny z intencją użytkownika.

Testowanie intuicyjne kładzie nacisk na walidację opartą na intencjach, skupiając się na testowaniu tego, co aplikacja powinna robić, a nie tylko tego, co obecnie robi, zamiast polegać na statycznych skryptach testowych. Zamiast skrupulatnie określać każdy przypadek testowy z wyprzedzeniem, testerzy wchodzą w interakcję z aplikacją tak, jak zrobiliby to prawdziwi użytkownicy, opierając się na testowaniu eksploracyjnym i przypadkach testowych generowanych przez AI. Testowanie intuicyjne przyspiesza proces testowy, sprawia, że testowanie jest bardziej elastyczne w stosunku do szybko ewoluujących aplikacji opartych na sztucznej inteligencji, zmniejsza zależność od manualnego pisania testów i ściśle wpisuje się w intuicyjny, wysokopoziomowy styl projektowania w programowaniu intuicyjnym. Jednak wiąże się to również z wyzwaniami. Ponieważ programiści mogą nie w pełni rozumieć lub nie przeglądać kodu źródłowego, istnieje ryzyko ukrytych defektów, luk w zabezpieczeniach, halucynacji AI lub niezrozumiałych zachowań. Testowanie intuicyjne łagodzi to ryzyko, działając jako zabezpieczenie i zapewniając, że aplikacja zapewnia zamierzoną funkcjonalność, a nie tylko to, co zinterpretowała sztuczna inteligencja.

Założmy na przykład, że programista użył modelu LLM do wygenerowania kodu z poleceniem „Utwórz stronę logowania z polami na adres e-mail i hasło. Po pomyślnym zalogowaniu użytkownicy powinni zostać przekierowani do tablicy wskaźników”. Tester wykorzystuje polecenia lub manualną eksplorację, aby przetestować zachowanie systemu z perspektywy użytkownika.

Przykłady poleceń obejmują: „Przetestuj logowanie przy użyciu nieprawidłowego formatu adresu e-mail i upewnij się, że nie nastąpi przekierowanie do tablicy wskaźników” lub „Po zalogowaniu się sprawdź, czy użytkownik widzi właściwą tablicę wskaźników w zależności od roli (np. administrator vs. zwykły użytkownik)”. Pierwotna instrukcja programisty nie określała reguł walidacji danych wejściowych, obsługi nieprawidłowych danych wejściowych ani logiki tablicy wskaźników opartej na rolach. Są to powszechne oczekiwania, ale jeśli nie zostaną wyraźnie określone, LLM może je pominąć. Testowanie intuicyjne ma na celu wyeliminowanie tych potencjalnych luk poprzez sprawdzenie, jak aplikacja powinna działać w rzeczywistych warunkach, zamiast zakładać, że LLM wszystko poprawnie zrozumiał na podstawie niejasnej instrukcji.

5.3. Zapachy testów

Zapach testu (ang. *test smell*) to symptom źle zaprojektowanego testu. Chociaż nie zawsze wskazuje na problem, sugeruje potencjalne ryzyko i zwiększone koszty utrzymania. Podobnie jak w tworzeniu oprogramowania, w celu utrzymania jakości kodu należy eliminować zapachy kodu (ang. *code smells*), tak samo należy eliminować zapachy testów, aby utrzymać skuteczną i godną zaufania strategię testów. Niniejszy sylabus opisuje wyłącznie zapachy testów w manualnych przypadkach testowych. W przypadku automatycznych przypadków testowych zobacz (Aljedaani i in., 2021).

Poniższe zapachy testów są pogrupowane w kategorie, z których każda zawiera opis zapachu testu oraz zalecane rozwiązanie:

Zapachy dotyczące zależności i złożoności

Testy współzależne (ang. *interdependent tests*)

- Zapach: Pomyślny wynik jednego testu zależy od tego, czy inny test zostanie wykonany wcześniej lub czy pozostawi resztkowe dane.
- Rozwiązanie: Spraw, by testy były niezależne, samowystarczalne i możliwe do wykonania w dowolnej kolejności.

Ukryte zależności (ang. *hidden dependencies*)

- Zapach: Przypadek testowy opiera się na zmianach danych lub konfiguracji, które nie są określone w warunkach wstępnych.
- Rozwiązanie: Spraw, by zależności były jawne i kontrolowane.

Wezwij mnie (ang. *call on me*)

- Zapach: Poleganie na rozbudowanym wywoływaniu innych przypadków testowych lub na przekazywaniu parametrów między nimi.
- Rozwiązanie: Unikaj zagnieżdżania wywołań i łańcuchowania przypadków testowych; zamiast tego dbaj o prostotę testów oraz stosuj jasne nazewnictwo i dokumentację.

Alternatywna wyrocznia (ang. *ORacle*)

- Zapach: Użycie logiki decyzyjnej w krokach testowych, np. oferowanie alternatywnych działań i wielu oczekiwanych wyników.
- Rozwiązanie: Spraw, by przypadki testowe były deterministyczne i pozbawione rozgałęzień poprzez podzielenie ich na oddzielne przypadki testowe, przeniesienie logiki decyzyjnej do warunków wstępnych oraz unikanie użycia alternatywy w oczekiwanych wynikach.

Zapachy dotyczące oczekiwanych wyników

Brak jasnych oczekiwanych wyników (ang. *no clear expected results*)

- Zapach: Procedury testowe wymieniają działania, ale nie definiują jasno kryteriów zaliczenia/niezaliczenia.
- Rozwiązanie: Dodaj oczekiwane wyniki, używając ustrukturyzowanego formatu, który sprawia, że oczekiwania są widoczne i możliwe do prześledzenia. Unikaj subiektywnego języka i zapewnij śledzenie powiązań w odniesieniu do wymagań lub kryteriów akceptacji.

Jeszcze jeden krok (ang. *one more step*)

- Zapach: Oczekiwane wyniki zawierają dodatkowe kroki testowe, często z użyciem czasowników określających czynność, takich jak „sprawdzić”, „zweryfikować” i „zobaczyć”.
- Rozwiązanie: Wyraźnie oddzielaj działania i oczekiwane wyniki od przebiegu testu, przenosząc weryfikację do osobnego kroku, oraz unikać używania czasowników określających czynność w oczekiwanych wynikach.

Zapachy dotyczące kroków testowych

Niejasne etapy (ang. *ambiguous steps*)

- Zapach: Instrukcje są niejasne lub pozostawiają pole do interpretacji.

- Rozwiązanie: Używaj precyzyjnego, obiektywnego języka z konkretnymi terminami i danymi. Jasno zdefiniuj dane wejściowe, unikaj subiektywnych słów i odwołuj się do konkretnych elementów interfejsu użytkownika lub etykiet.

Skoczek (ang. *hotstepper*)

- Zapach: Lista kroków testowych wymienia każdą czynność osobno, nawet jeśli nie ma ona żadnego efektu.
- Rozwiązanie: Połącz kroki, które z osobna nie mają konkretnego efektu.

Kroki zbiorcze (ang. *bulk steps*)

- Zapach: Wiele działań jest zawartych w jednym kroku testowym.
- Rozwiązanie: W stosownych przypadkach stosuj warunki wstępne i przestrzegaj zasady „jedna czynność = jeden krok”, upewniając się, że do każdej czynności dodano odpowiednie oczekiwane wyniki.

Zapachy związane z poziomem szczegółowości

Zbyt długi lub szczegółowy przypadek testowy

- Zapach: Przypadki testowe z wieloma krokami testowymi dotyczącymi wielu funkcji lub przypadki testowe, które zawierają pełną ścieżkę nawigacji przed każdą małą czynnością.
- Rozwiązanie: Podziel na mniejsze przypadki testowe, zmodularyzuj części wielokrotnego użytku, usuń powtarzające się szczegóły nawigacji i efektywnie stosuj warunki wstępne.

Boski przypadek testowy (ang. *god test case*)

- Zapach: Pojedynczy przypadek testowy jest używany do testowania wielu alternatywnych scenariuszy, a nawet całego systemu.
- Rozwiązanie: Zaprojektuj małe, ukierunkowane, niezależne przypadki testowe, zorganizowane w zestawy testowe.

Dane testowe zakodowane na stałe (ang. *hardcoded test data*)

- Zapach: Każdy przypadek testowy jawnie zawiera wszystkie dane testowe.
- Rozwiązanie: W stosownych przypadkach stosuj opisy danych wysokiego poziomu oraz używaj parametrów w celu oddzielenia danych testowych od logiki przypadku testowego.

Zapachy braków

Założenia dotyczące środowiska (ang. *environment assumptions*)

- Zapach: Przypadek testowy zakłada określone warunki środowiska testowego bez ich dokumentowania.
- Rozwiązanie: Wyraźnie udokumentuj warunki wstępne środowiska i wymagania dotyczące konfiguracji.

Brak czyszczenia i rozmontowania (ang. *no cleanup or teardown*)

- Zapach: Przypadek testowy nie określa, w jaki sposób dane testowe są obsługiwane po zakończeniu testu.
- Rozwiązanie: Zdefiniuj wyraźne procedury czyszczenia lub rozmontowania środowiska testowego po wykonaniu testu.

Zapachy nadmiaru

Zalew mechanizmów unieważnień (ang. *invalidation heaven*)

- Zapach: Każdy typ nieprawidłowych danych wejściowych jest obsługiwany przez oddzielny przypadek testowy.
- Rozwiązanie: Połącz obsługę nieprawidłowych danych wejściowych w jeden przypadek testowy, używając parametrów.

Duplikacja typu „kopiuj-wklej” (ang. *copy-paste duplication*)

- Zapach: Wiele przypadków testowych różni się jedynie kilkoma wartościami danych testowych.
- Rozwiązanie: Przeprojektuj kod w jeden, sparametryzowany przypadek testowy wielokrotnego użytku, aby zapobiec powielaniu, jednocześnie zapewniając śledzenie powiązań względem wymagań w celu pomiaru pokrycia.

Zapachy związane z formatowaniem i językiem

Nieuporządkowane formatowanie (ang. *unstructured formatting*)

- Zapach: Kroki testowe nie są ponumerowane i nie mają nagłówków.
- Rozwiązanie: Użyj szablonów.

Literówki (ang. *typos*)

- Zapach: Przypadki testowe zawierają błędy ortograficzne oraz niekompletne słowa lub zdania.
- Rozwiązanie: Użyj programu sprawdzającego pisownię.

Kliknij-Naciśnij-Wciśnij (ang. *click-push-press*)

- Zapach: Terminologia i sformułowania używane w przypadkach testowych są niespójne.
- Rozwiązanie: Korzystaj ze słownika standardowych wyrażen.

6. Automatyzacja testów i narzędzia testowe (30 min.)

Słowa kluczowe

automatyzacja testów, zwinne wytwarzanie oprogramowania

Cele nauczania dla rozdziału 6

6.1 Automatyzacja testów w zwinnym wytwarzaniu oprogramowania

AT-6.1.1 (K2) Rozróżniać podejścia do automatyzacji testów stosowane w zwinnym wytwarzaniu oprogramowania.

6.2 Narzędzia testowe w zwinnym wytwarzaniu oprogramowania

AT-6.2.1 (K2) Podać przykłady narzędzi testowych pomocnych w testowaniu zwinnym.

Wstęp

Automatyzacja testów może zapewnić szybką i wiarygodną informację zwrotną w krótkich cyklach testowych. Automatyzacja testów uzupełnia, ale nie zastępuje testowania manualnego, w tym eksploracyjnego. Narzędzia testowe wspomagają współpracę, śledzenie wymagań, monitorowanie wydajności i raportowanie testów. Narzędzia, które płynnie integrują się ze zwinnym wytwarzaniem oprogramowania, poprawiają przejrzystość, przyspieszają informację zwrotną i pomagają całemu zespołowi dzielić się odpowiedzialnością za jakość.

6.1. Automatyzacja testów w zwinnym wytwarzaniu oprogramowania

Decyzja o automatyzacji testów wynika zarówno z ryzyka, jak i potencjalnych korzyści. Automatyzacja testów dobrze sprawdza się w przypadku zadań, które są powtarzalne, deterministyczne i mają dużą wartość pod względem redukcji ryzyka, zwłaszcza gdy awarie mogą wpłynąć na działanie krytycznych funkcji biznesowych. Testy, które nigdy nie zostaną niezaliczone, mają niewielką wartość i oferują minimalne korzyści po zautomatyzowaniu. Podobnie testy, których zachowanie często się zmienia, nie nadają się do automatyzacji, ponieważ koszty utrzymania często przewyższają korzyści.

Za najlepszą praktykę uważa się podjęcie decyzji podczas planowania lub dopracowywania iteracji, które historyjki użytkownika należy zautomatyzować i w jakim zakresie. Ułatwia to dokładniejsze oszacowanie czasu potrzebnego do pełnej akceptacji historyjki użytkownika, co skutkuje większą przewidywalnością realizacji iteracji.

W zwinnym wytwarzaniu oprogramowania stosuje się różne podejścia do automatyzacji testów w zależności od kontekstu, umiejętności zespołu, krytyczności systemu oraz częstotliwości dostaw. Jednym z powszechnych podejść jest automatyzacja testów, które zapewniają szybką informację zwrotną i powtarzalną walidację najbardziej wartościowych i stabilnych części systemu. Zazwyczaj obejmuje to testy modułowe, testy API oraz starannie dobrany zestaw testów kompleksowych, zgodnie z piramidą testów (ISTQB-CTFL, w. 4.0.1). Testy eksploracyjne i testy jednorazowe (tj. testy wykonywane tylko raz w celu sprawdzenia konkretnego warunku testowego w danym momencie) są zazwyczaj przeprowadzane manualnie, ponieważ opierają się na ludzkiej intuicji, są napędzane przez zachowania emergentne lub koszty ich utrzymania byłyby zbyt wysokie. Te same względy często mają zastosowanie do testów użyteczności.

Kluczowym czynnikiem jest również moment wprowadzenia automatyzacji testów. W podejściu „najpierw test” testy są automatyzowane wcześniej, przed lub w trakcie pisania kodu. Pozwala to na wytwarzanie sterowane testami i zapewnia ciągłą weryfikację. Natomiast automatyzacja testów regresji jest często działaniem, które ewoluuje w trakcie lub po iteracji, w zależności od tego, kiedy funkcjonalność się ustabilizuje.

Testowanie zwinne integruje automatyzację testów w całej iteracji, zaczynając od weryfikacji kompilacji w ramach ciągłej integracji, poprzez testy eksploracyjne wspierane przez narzędzia do automatyzacji testów, a kończąc na zautomatyzowanych zestawach testów regresji, które potwierdzają stabilność systemu przed wydaniem.

Skuteczna automatyzacja testów w zwinnym wytwarzaniu oprogramowania wymaga wielowarstwowej strategii, która koncentruje się na automatyzacji tego, co zapewnia spójną, wartościową informację zwrotną, wyborze odpowiednich testów do automatyzacji oraz stopniowej integracji automatyzacji testów w całym cyklu życia oprogramowania.

Piramida testów w metodykach zwinnych, wprowadzona przez Mike'a Cohna, promuje zrównoważoną strategię testów poprzez priorytetowe traktowanie szybkich testów modułowych

niskiego poziomu, wspieranie ich mniejszą warstwą testów usług i integracji oraz utrzymywanie jedynie minimalnego zestawu kompleksowych testów interfejsu użytkownika, co umożliwia szybką informację zwrotną, obniżenie kosztów i większą stabilność CI/CD.

Inne sylabusy ISTQB zapewniają bardziej dogłębną wiedzę na temat automatyzacji testów (patrz: (ISTQB-TAE, v.2.0), (ISTQB-TAS, v.1.0)).

6.2. Narzędzia testowe w zwinnym wytwarzaniu oprogramowania

Narzędzia testowe stosowane w testowaniu zwinnym obejmują wiele kategorii i wspierają zarówno iteracyjny, jak i oparty na współpracy charakter zwinnego wytwarzania oprogramowania. Zespoły zwinne czerpią korzyści z narzędzi, które ułatwiają komunikację, ciągłą informację zwrotną i automatyzację testów w całym cyklu życia oprogramowania. Wybór narzędzi testowych powinien być dostosowany do strategii testów, warstw automatyzacji testów i celów informacji zwrotnej, a nie do wygody czy trendów.

Różnorodne narzędzia wspierają działania związane z testowaniem zwinnym oraz współpracę w zespole, a każde z nich służy określonej roli w ramach procesu wytwarzania i dostarczania oprogramowania:

Narzędzia do zarządzania zadaniami i śledzenia umożliwiają zespołowi planowanie, ustalanie priorytetów i śledzenie historii użytkownika, zadań testowych i defektów, wspierając przejrzystość i wspólne zrozumienie postępów zespołu oraz ryzyk. Narzędzia te są niezbędne do utrzymywania backlogu, zarządzania defektami i zarządzania kartami opisu testu.

Narzędzia do komunikacji i dzielenia się informacjami wspierają współpracę w czasie rzeczywistym, zwłaszcza w zespołach rozproszonych. Usprawniają one interakcję podczas sesji dopracowywania, planowania iteracji, przeglądów testów i retrospektyw. Narzędzia te pomagają testerom dzielić się spostrzeżeniami, omawiać defekty i szybko przekazywać informacje zwrotne od klientów całemu zespołowi.

Narzędzia do projektowania, wdrażania i wykonywania testów wspierają tworzenie i wykonywanie zarówno testów automatycznych, jak i manualnych. Obejmują one struktury do automatyzacji testów modułowych, testów API oraz testów GUI. Wspomagają one również praktyki specyfikacji przez przykłady, takie jak ATDD i BDD, ułatwiając współpracę między członkami zespołu w zakresie kryteriów akceptacji.

Narzędzia do ciągłej integracji umożliwiają częstą integrację kodu i szybkie wykrywanie awarii, jednocześnie zapewniając stałą gotowość oprogramowania do wdrożenia. Narzędzia te są niezbędne w zwinnym wytwarzaniu oprogramowania, gdzie krótkie pętle informacji zwrotnej wspierają iteracyjne dostarczanie i pomagają wcześniej łagodzić ryzyka regresji.

Narzędzia do zarządzania konfiguracją zapewniają spójność między środowiskami testowymi poprzez zarządzanie kodem, skryptami testowymi i zmianami infrastruktury. Narzędzia te są często zintegrowane z potokami CI/CD w celu wspierania celów DevOps i ciągłego dostarczania.

Narzędzia do testowania eksploracyjnego wspierają zarządzanie testowaniem w sesjach, śledzenie kart opisu testu, rejestrowanie arkuszy sesji testowych oraz wykonywanie zrzutów ekranu. Narzędzia te pomagają rejestrować warunki testowe i wyniki testów przy minimalnym nakładzie pracy związanym z dokumentacją.

Narzędzia do monitorowania i analizy zapewniają wgląd na poziomie produkcyjnym poprzez śledzenie wzorców użytkowania, wskaźników defektów i metryk wydajności. Są one szczególnie przydatne w zespołach zwinnych, które stosują monitorowanie jako testowanie, oraz w tych, które wykorzystują rzeczywiste opinie użytkowników do udoskonalania zachowania produktu po wdrożeniu.

Narzędzia AI mogą automatycznie generować przypadki testowe na podstawie zmian w kodzie, wymagań lub wzorców zachowań użytkowników; przewidywać awarie kompilacji na podstawie zmian w kodzie i danych historycznych; analizować historyjki użytkownika lub wymagania w celu wykrycia niejasności, brakujących kryteriów akceptacji lub niespójnej terminologii oraz tworzyć realistyczne dane testowe obejmujące przypadki skrajne.

7. Bibliografia

Sylabusy ISTQB®

ISTQB-ATLAS, ISTQB®, 2023. *Certified Tester Agile Test Leadership at Scale: Syllabus*. Version 1.0.

ISTQB-CTFL, ISTQB®, 2024. *Certified Tester Foundation Level: Syllabus*. Version 4.0.1.

ISTQB-TAE, ISTQB®, 2024. *Certified Tester Test Automation Engineering: Syllabus*. Version 2.0.

ISTQB-TAS, ISTQB®, 2024. *Certified Tester Test Automation Strategy: Syllabus*. Version 1.0.

(Polskie wersje niektórych z powyższych sylabusów znajdują się na stronie www.pqb.org.pl)

Słowniki pojęć

ISTQB® Glossary: <https://glossary.istqb.org>

Agile Alliance: <https://agilealliance.org/agile101/agile-glossary>

IEEE/Pascal for software engineering: https://pascal.computer.org/sev_display/index.action

IREB-CPRE for Requirements Engineering: <https://cpre.ireb.org/en/downloads-and-resources/glossary>

Normy

IEEE 1044 – *Classification for Software Anomalies*, 2009. Standard. Institute of Electrical and Electronics Engineers.

ISO 29119-6 *Software and systems engineering – Software testing: Part 6: Guidelines for the use of ISO/IEC/IEEE 29119 (all parts) in agile projects*, 2021. Technical report. International Organization for Standardization.

Książki

ADZIC, Gojko, 2011. *Specification by Example: How Successful Teams Deliver the Right Software*. Manning.

ANDERSON, D.J.; REINERSTEN, D.G., 2010. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press.

ANDERSON, L.W.; KRATHWOHL, D.R., 2001. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman. ISBN 9780801319037.

BECK, K.; ANDRES, C., 2004. *Extreme Programming Explained: Embrace Change*, Wyd. 2.. Addison-Wesley.

BEIZER, Boris, 1990. *Software Testing Techniques*. Van Nostrand Reinhold: Boston MA.

GREAVES, K.; LAING, S., 2019. *Growing Agile: A Coach's Guide to Agile Testing*. Leanpub.

GREGORY, J.; CRISPIN, L., 2019. *Agile Testing Condensed: A Brief Introduction*. Leanpub.

WHITTAKER, J.A., 2009. *Exploratory Software Testing: Tips, Tricks, Tours, and Techniques to Guide Test Design*. Addison-Wesley Professional.

Artykuły

ALJEDAANI, Wajdi; PERUMA, Anthony; ALJOHANI, Ahmed; ALOTAIBI, Mazen; MKAOUER, Mohamed Wiem; OUNI, Ali; NEWMAN, Christian D.; GHALLAB, Abdullatif; LUDI, Stephanie, 2021. Test Smell Detection Tools: A Systematic Mapping Study, ss. 170–180. Available from DOI: 10.1145/3463274.3463335.

GHAZI, Ahmad Nauman; GARIGAPATI, Ratna Pranathi; PETERSEN, Kai, 2017. Checklists to Support Test Charter Design in Exploratory Testing, pp. 251–258. ISBN 978-3-319-57633-6.

NIELSEN, Jakob, 1994. *Enhancing the explanatory power of usability heuristics. Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, ss. 152–158.

WALKER, R.; CENYDD, L. ap; POP, S.; MILES, H. C.; HUGHES, C. J.; TEAHAN, W. J.; ROBERTS, J. C., 2013. Storyboarding for visual analytics. *Information Visualization*. Vol. 14, ss. 27–50.

Źródła internetowe

Agile Alliance. <https://agilealliance.org/agile101/agile-glossary>.

BECK, K.; BEEDLE, M.; BENNEKUM, A. van; COCKBURN, A.; CUNNINGHAM, W.; FOWLER, M.;

GRENNING, J.; HIGHSMITH, J.; HUNT, A.; JEFFRIES, R.; KERN, J.; MARICK, B.; MARTIN, R.;

MELLOR, S.; SCHWABER, K.; SUTHERLAND, J.; THOMAS, D., 2001. Agile Manifesto [online]. [data dostępu 2025-08-30]. Dostępne z: <https://agilemanifesto.org/>.

BOLTON, M., 2012. FEW HICCUPS [online]. [data dostępu 2025-07-27]. Dostępne z: <https://developsense.com/blog/2012/07/few-hiccups>.

BOLTON, M., 2014. How Models Change [online]. [data dostępu 2025-07-27]. Dostępne z: <https://developsense.com/blog/2014/07/how-models-change>.

GAREEV, A., 2019. The TERMS for Test Automation [online]. [data dostępu 2025-07-27]. Dostępne z: <https://www.automation-beyond.com/2019/04/10/the-terms-for-test-automation>.

GREGORY, J., 2021. Testing from a holistic point of view [online]. [visited on 2025-09-05]. Dostępne z: <https://janetgregory.ca/testing-from-a-holistic-point-of-view/>.

IEEE/Pascal for software engineering. Dostępne także z: https://pascal.computer.org/sev_display/index.action.

IREB-CPRE for Requirements Engineering. Dostępne także z: <https://cpre.ireb.org/en/downloadsand-resources/glossary>.

JOHNSON, K., 2012. Software Testing Heuristics and Mnemonics [online]. [data dostępu 2025-07-27]. Dostępne z: <https://www.slideshare.net/slideshow/kn-johnson-2012-heuristics-mnemonics/15019733>.

KOHL, J., 2010. Test Mobile Applications with I SLICED UP FUN! [online]. [data dostępu 2025-07-27]. Dostępne z: <https://www.kohl.ca/articles/ISLICEDUPFUN.pdf>.

OWASP, 2025. Top Ten Web Application Security Risks [online]. [data dostępu 2025-07-27]. Dostępne z: <https://owasp.org/www-project-top-ten>.

PROSCI, 2026. The Prosci ADKAR Model [online]. [data dostępu 2026-01-04]. Dostępne z: <https://www.prosci.com/methodology/adkar>.

SUTHERLAND, J.; SCHWABER, K., 2020. Scrum Guide [online]. [data dostępu 2025-08-30]. Dostępne z: <https://scrumguides.org/index.html>.

WYNNE, M., 2015. Introducing Example Mapping [online]. [data dostępu 2025-10-07]. Dostępne z: <https://cucumber.io/blog/bdd/example-mapping-introduction>.

Powyższe odniesienia wskazują na informacje dostępne w Internecie i innych miejscach. Mimo że odniesienia te zostały sprawdzone w momencie publikacji niniejszego sylabusu, ISTQB® nie ponosi odpowiedzialności, jeśli odniesienia te są już niedostępne.

8. Pozostała literatura (niecytowana)

GREGORY, J.; CRISPIN, L., 2024. *Holistic Testing: Weave Quality into Your Product*. Leanpub.

HENDRICKSON, Elisabeth, 2013. *Explore It!* Pragmatic Bookshelf.

ISO 29119-1 *Software and systems engineering – Software testing: Part 1: General concepts*, 2022. Standard. International Organization for Standardization.

ISO/IEC 20246: *Software and systems engineering – Work product reviews*, 2017. Standard. International Organization for Standardization.

ISO/IEC 25010: *Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*, 2023. Standard. International Organization for Standardization.

ISO/IEC 25019: *Systems and software Quality Requirements and Evaluation (SQuaRE) – Quality-in-use model*, 2023. Standard. International Organization for Standardization.

ISO/IEC/IEEE 29119-3: *Software testing, Part 3: Test documentation*, 2021. Standard. International Organization for Standardization.

ISO/IEC/IEEE 29119-4: *Software testing, Part 4: Test techniques*, 2021. Standard. International Organization for Standardization.

ISO/IEC/IEEE 29119-5: *Software testing, Part 5: Keyword-driven testing*, 2016. Standard. International Organization for Standardization.

ISTQB-AI, ISTQB®, 2021. *Certified Tester AI Testing: Syllabus*. Version 1.0.

ISTQB-ITP, ISTQB®, 2011. *Certified Tester Expert Level Improving the Test Process: Syllabus*. Version 1.0.

ISTQB-MBT, ISTQB®, 2024. *Certified Tester Model-Based Tester: Syllabus*. Version 1.1.

ISTQB-PT, ISTQB®, 2018. *Certified Tester Performance Testing: Syllabus*. Version 1.0.

ISTQB-TM, ISTQB®, 2024. *Certified Tester Advanced Level Test Management: Syllabus*. Version 3.0.

ISTQB-TTA, ISTQB®, 2021. *Certified Tester Advanced Level Technical Test Analyst: Syllabus*. Version 4.0.

ISTQB-UT, ISTQB®, 2018. *Certified Tester Usability Testing: Syllabus*. Version 1.0.

9. Dodatek A – lista skrótów

Skrót	Znaczenie	Polskie tłumaczenie
AI	Artificial Intelligence	sztuczna inteligencja
API	Application Programming Interface	interfejs programowy aplikacji
ATDD	Acceptance Test-Driven Development	wytwarzanie sterowane testami akceptacyjnymi
BDD	Behavior-Driven Development	wytwarzanie sterowane zachowaniem
BPMN	Business Process Model and Notation	Notacja i Model Procesu Biznesowego
CI/CD	Continuous Integration/Continuous Delivery	ciągła integracja/ciągłe dostarczanie
GQM	Goal-Question-Metric	Cel-Pytanie-Metryka
LLM	Large Language Model	duży model językowy
MTTR	Mean Time to Recovery	średni czas do naprawy
OWASP	Open Web Application Security Project	otwarty projekt zabezpieczeń aplikacji webowych
TDD	Test-Driven Development	wytwarzanie sterowane testami
UI	User Interface	interfejs użytkownika
XP	eXtreme Programming	programowanie ekstremalne

10. Dodatek B – słowa kluczowe specyficzne dla dziedziny

Termin	Definicja
arkusz sesji testowej	Dokument wykorzystywany w testowaniu eksploracyjnym do rejestrowania przebiegu i wyników sesji testowej o określonym czasie trwania, w tym notatek dotyczących przeprowadzonych czynności testowych, spostrzeżeń, osiągniętego pokrycia testowego, wykrytych defektów oraz działań następnych.
Cel-Pytanie-Metryka	Podejście do pomiarów, które określa cele, formułuje pytania służące do oceny realizacji tych celów oraz określa metryki pozwalające odpowiedzieć na te pytania w sposób systematyczny i umożliwiający śledzenie wyników.
ciche wdrożenie	Praktyka wdrażania nowych lub zmienionych funkcji do środowiska produkcyjnego bez udostępniania ich użytkownikom końcowym, w celu sprawdzenia zachowania, wydajności lub ryzyka w rzeczywistych warunkach przed pełnym uruchomieniem.
definicja ukończenia	Formalny opis stanu przyrostu, gdy spełnia on wymagane kryteria jakości produktu.
FEW HICCUPPS	Heurystyka testowa uwzględniająca znajomość, wyjaśnialność, świat, historię, obraz, produkty porównywalne, twierdzenia, pragnienia użytkowników, produkt, cel, przepisy, mająca na celu wspieranie identyfikacji wyroczeni testowych.
historyjka użytkownika	Krótką relacją przedstawiającą potrzebę z perspektywy użytkownika wraz z oczekiwanymi korzyściami wynikającymi z jej zaspokojenia.
I SLICED UP FUN	Heurystyka testowa obejmująca dane wejściowe, przechowywanie, lokalizację, interakcje/przerwania, komunikację, ergonomię, dane, użyteczność, platformę, funkcję, scenariusze użytkownika, sieć.
iteracja	Cykl o stałej długości i ograniczonym czasie, w ramach którego realizowany jest zestaw zaplanowanych działań w celu stworzenia wersji produktu gotowej do potencjalnego wydania.
iteracja wzmacniająca	Iteracja, zazwyczaj przeprowadzana na końcu serii iteracji programistycznych, której głównym celem jest ustabilizowanie produktu poprzez usunięcie pozostałych defektów, poprawę wydajności, zakończenie działań związanych z integracją i testowaniem systemowym oraz przygotowanie produktu do wydania.
mapowanie przykładów	Technika oparta na współpracy stosowana w zwinnym wytwarzaniu oprogramowania, wykorzystująca reguły, przykłady i pytania w celu wyjaśnienia wymagań i wsparcia projektowania testów.
metryka	Skala pomiarowa i metoda stosowana do pomiaru.
opowieść	Opis potrzeby interesariusza, który zazwyczaj wykracza poza to, co można zrealizować w ramach pojedynczej iteracji.
ścieżka użytkownika	Przedstawienie sekwencji kroków, które użytkownik wykonuje, aby osiągnąć określony cel podczas interakcji z systemem, w tym doświadczenia użytkownika, jego odczucia i reakcje emocjonalne na każdym etapie.
podział historyjki użytkownika	Praktyka polegająca na rozbiciu dużej lub złożonej historyjki użytkownika na mniejsze, łatwiejsze do opanowania i możliwe do przetestowania

	historyjki użytkownika.
przełącznik funkcji	Mechanizm służący do modyfikowania zachowania systemu bez zmiany kodu.
RCRCRC	Heurystyka testowa dla testowania regresji obejmująca: <i>Recent</i> (ostatnie), <i>Core</i> (kluczowe), <i>Risky</i> (ryzykowne), <i>Configuration</i> (konfiguracja), <i>Repaired</i> (naprawione), <i>Chronic</i> (przewlekłe).
SFDIPOT	Heurystyka testowa dotycząca struktury, funkcji, danych, interfejsów, platformy, utrzymania i czasu.
specyfikacja przez przykłady	Technika programistyczna, w której specyfikacja jest definiowana za pomocą przykładów.
społeczność praktyków	Grupa specjalistów, których łączą wspólne zainteresowania, pasja lub wyzwania, spotykająca się regularnie w celu dzielenia się wiedzą, uczenia się na podstawie wzajemnych doświadczeń oraz doskonalenia umiejętności w zakresie rozwiązywania problemów i wprowadzania innowacji w swojej konkretnej dziedzinie.
<i>storyboarding</i>	Wizualna prezentacja sekwencji interakcji użytkownika lub zachowań systemu, wykorzystywana do wyjaśnienia wymagań, wsparcia projektowania testów oraz skutecznego komunikowania scenariuszy.
stronniczość	Systematyczne odejście od obiektywności, które może wpływać na cele testów, czynności testowe lub wyniki testów.
TERMS	Heurystyka testowa w zakresie narzędzi i technologii, wykonywania testów, wymagań i ryzyk, pielęgnacji i zabezpieczeń.
ustrukturyzowana rozmowa	Moderowana, ukierunkowana dyskusja, która przebiega zgodnie z określoną strukturą lub zestawem instrukcji, aby zapewnić wspólne zrozumienie, skupienie się na celach oraz skuteczną wymianę informacji między interesariuszami.
warsztaty ryzyka	Ustrukturyzowana technika burzy mózgów, w ramach której Interesariusze wspólnie identyfikują potencjalne ryzyka w celu wsparcia testowania opartego na ryzyku.
wydanie kanarkowe	Strategia wdrażania oprogramowania, w ramach której nowa wersja systemu jest udostępniana ograniczonej grupie użytkowników w celu jej oceny przed szerszym wdrożeniem dla wszystkich użytkowników.

11. Dodatek C – cele nauczania i poziomy wiedzy

Szczegółowe cele nauczania dotyczące niniejszego sylabusu są przedstawione na początku każdego rozdziału. Każdy temat opisany w sylabusie będzie objęty egzaminem zgodnie z jego celem nauczania. Cele nauczania rozpoczynają się od czasownika określającego czynność odpowiadającą wymaganemu poziomowi wiedzy, zgodnie z poniższym wykazem.

Poziom 1: zapamiętać (K1)

Kandydat pamięta, rozpoznaje lub przypomina sobie dany termin lub dane pojęcie.

Czasowniki określające czynność: przypomnieć sobie, rozpoznać.

Uwaga: sylabusy na poziomie zaawansowanym nie zawierają konkretnych celów nauczania na poziomie K1. Jednakże treść sylabusu zawarta w rozdziałach 1–5 oraz definicje wszystkich terminów wymienionych jako słowa kluczowe tuż pod nagłówkami rozdziałów powinny zostać zapamiętane (K1), nawet jeśli nie zostały one wyraźnie wymienione w celach nauczania.

Przykłady:

Przypomnieć sobie koncepcję piramidy testów.

Rozpoznać typowe cele testowania.

Poziom 2: zrozumieć (K2)

Kandydat potrafi uzasadnić lub wyjaśnić stwierdzenia dotyczące danego zagadnienia, a także podsumować, porównać, sklasyfikować i podać przykłady pojęć z zakresu testowania.

Czasowniki określające czynność: sklasyfikować, porównać, rozróżniać, odróżnić, wyjaśnić, podać przykłady, zinterpretować, podsumować.

Przykłady	Uwagi
Sklasyfikować narzędzia testowe według ich przeznaczenia i czynności testowych, które wspierają.	
Porównać różne poziomy testów.	Może służyć do wyszukiwania podobieństw, różnic lub obu naraz.
Odróżniać testowanie od debugowania.	Szukać różnic pomiędzy koncepcjami.
Rozróżniać ryzyka projektowe i ryzyka produktowe.	Pozwala na oddzielne zaklasyfikowanie dwóch (lub więcej) koncepcji.
Wyjaśnić wpływ kontekstu na proces testowy.	
Podać przykłady, dlaczego testowanie jest niezbędne.	
Wynioskować pierwotną przyczynę defektów na podstawie danego profilu awarii.	
Podsumować czynności procesu przeglądu produktu pracy.	

Poziom 3: zastosować (K3)

Kandydat potrafi wykonać odpowiednią procedurę, gdy zostanie mu postawione znane mu zadanie bądź wybrać poprawną procedurę i zastosować ją w danym kontekście.

Czasowniki określające czynność: zastosować, wdrożyć, przygotować, użyć.

Przykłady	Uwagi
Zastosować analizę wartości brzegowych do wyprowadzenia przypadków testowych z danych wymagań.	Powinno odnosić się do procedury, techniki, procesów itp.
Wdrożyć metody zbierania metryk w celu wsparcia technicznego i i zarządzania wymaganiami.	
Przygotować testy instalowalności dla aplikacji mobilnej.	Może być użyte w celu nauczania, który każe kandydatowi użyć techniki lub procedury. Podobne do „zastosować”.
Wykorzystać śledzenie powiązań do monitorowania postępu testów w zakresie kompletności i spójności z celami testów, strategią testów i planem testów.	Może być użyte w celu nauczania, który wymaga od kandydata wykorzystania techniki lub procedury. Podobne do „zastosować”.

Poziom 4: przeanalizować (K4)

Kandydat potrafi rozdzielić informacje związane z procedurą lub techniką na poszczególne części składowe w celu lepszego zrozumienia oraz potrafi odróżnić fakty od wniosków. Typowym zastosowaniem jest analiza dokumentu, oprogramowania lub sytuacji projektowej oraz zaproponowanie odpowiednich działań w celu rozwiązania problemu lub wykonania zadania.

Czasowniki: określające czynność: przeanalizować, zdekonstruować, nakreślić, ustalić priorytety, wybrać.

Przykłady	Uwagi
Przeanalizować daną sytuację projektową w celu określenia, które czarnoskrzynkowe techniki testowania czy techniki testowania oparte na doświadczeniu powinny być zastosowane, aby osiągnąć określone cele.	Podlegające egzaminowaniu tylko w połączeniu z mierzalnym celem analizy. Powinno mieć formę „przeanalizuj X w celu Y” (lub podobną).
Ustalić priorytety przypadków testowych w danym zestawie testowym do wykonania w oparciu o powiązane ryzyka produktowe.	Potrzebne złagodzenia, kiedy wybór wymaga analizy.
Wybrać odpowiednie poziomy testów i typy testów w celu weryfikacji danego zestawu wymagań.	Potrzebne, gdy wybór wymaga analizy.

Materiały dodatkowe w zakresie poziomów poznawczych celów nauczania:

Anderson, L. W., Krathwohl, D. R. (red.), 2001, *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*, Allyn & Bacon.

12. Dodatek D – macierz powiązań między celami biznesowymi a celami nauczania

Cele biznesowe – poziom zaawansowany tester zwinny		BO1	BO2	BO3	BO4	BO5	BO6	BO7	BO8
BO1	Współpracować w zespołach wielofunkcyjnych, znając zasady i podstawowe praktyki zwinnego wytwarzania oprogramowania.	6							
BO2	Dostosować dotychczasowe doświadczenie i wiedzę z zakresu testowania do wartości i zasad Agile.		8						
BO3	Wspierać zespół zwinny w planowaniu testów.			6					
BO4	Stosować odpowiednie podejścia do wytwarzania oprogramowania w metodologii Agile oraz techniki testowania w celu zapewnienia testów o odpowiednim pokryciu.				10				
BO5	Pomóc interesariuszom biznesowym w definiowaniu zrozumiałych i możliwych do przetestowania historyjek użytkownika, scenariuszy, wymagań i kryteriów akceptacji, w zależności od potrzeb.					6			
BO6	Tworzyć i wdrażać różne podejścia do testowania w zwinnym wytwarzaniu oprogramowania.						10		
BO7	Wspierać i przyczyniać się do automatyzacji testów w projekcie zwinnego wytwarzania oprogramowania.							2	
BO8	Współpracować z innymi członkami zespołu oraz dzielić się z nimi informacjami przy użyciu skutecznych stylów i kanałów komunikacji.								9

Cele nauczania – Poziom Zaawansowany Tester Zwinny	Poziom K	BO1	BO2	BO3	BO4	BO5	BO6	BO7	BO8
1. Wyzwania związane ze strategią testów i podejściem do testów (60 min.).									
1.1 Typy testów									
AT-1.1.1 Porównać typy testów wykonywane w trakcie i po iteracji.	K2		X	X					
1.2 Testowanie kompleksowe									
AT-1.2.1 Wyjaśnić, kiedy powinno być wykonywane testowanie kompleksowe.	K2		X	X					
1.3 Testowanie formalne i testowanie holistyczne									
AT-1.3.1 Porównać korzyści i wady testowania formalnego i testowania holistycznego.	K2		X						
1.4 Podejścia do testowania regresji									
AT-1.4.1 Rozróżniać podejścia do testowania regresji.	K2		X	X			X		
2. Ludzie i zespoły (60 min.)									
2.1 Podejście „cały zespół”									
AT-2.1.1 Porównać generalizację i specjalizację w ramach zespołu.	K2	X							X
AT-2.1.2 Podać przykłady motywowania interesariuszy biznesowych do udziału w testach.	K2	X							X

Cele nauczania – Poziom Zaawansowany Tester Zwinny	Poziom K	B01	B02	B03	B04	B05	B06	B07	B08
AT-2.1.3 Podsumować, jak podejście „cały zespół” może wspomagać zespół wytwórczy.	K2	X							X
2.2 Testerzy jednorazowi									
AT-2.2.1 Wyjaśnić, jak i kiedy wykorzystywać testerów jednorazowych.	K2				X				
3. Zarządzanie testami i doskonalenie procesu testowego (210 min.)									
3.1 Planowanie testów									
AT-3.1.1 Podsumować planowanie testowania w zwinnym wytwarzaniu oprogramowania.	K2	X	X	X					
AT-3.1.2 Nakreślić projektową strategię testową używając kwadrantów testowych.	K4		X				X		
3.2 Monitorowanie testów i nadzór nad testami									
AT-3.2.1 Wyjaśnić, jak monitorować testy i sprawować nadzór nad testami w zwinnym wytwarzaniu oprogramowania.	K2			X					
3.3 Raportowanie testów									
AT-3.3.1 Porównać różne typy pokrycia, które można użyć do raportowania testów w zwinnym wytwarzaniu oprogramowania.	K2		X		X				
3.4 Doskonalenie procesu testowego									
AT-3.4.1 Wybrać odpowiednie działania mające na celu doskonalenie procesu testowego w oparciu o wskaźniki stosowane w zwinnym wytwarzaniu oprogramowania.	K4	X	X						X
AT-3.4.2 Wyjaśnić, jak przeprowadzić doskonalenie procesu testowego w zwinnym wytwarzaniu oprogramowania.	K2	X		X					X
4. Przesunięcie w lewo (135 min.)									
4.1 Użycie przesunięcia w lewo w celu zwiększenia jakości podstawy testów									
AT-4.1.1 Podać przykłady tego, jak testalia mogą być użyte jako wymagania.	K2					X			X
AT-4.1.2 Wyjaśnić, jak <i>storyboarding</i> i <i>testboarding</i> mogą zwiększyć jakość podstawy testów.	K2					X			X
AT-4.1.3 Wyjaśnić, jak mapowanie przykładów może zwiększyć jakość podstawy testów.	K2					X			X
AT-4.1.4 Podać przykłady tego, jak stroniczości mogą negatywnie wpłynąć na jakość produktu.	K2					X			
AT-4.1.5 Zastosować podział historyjki użytkownika w celu uzyskania testowalnych historyjek użytkownika.	K3					X			
4.2 Przesunięcie w lewo i inżynieria wymagań									
AT-4.2.1 Wyjaśnić, jak inżynieria wymagań wspiera przesunięcie w lewo.	K2					X			
5. Zwinne podejścia do testów i techniki testowania (285 min.)									
5.1 Testowanie eksploracyjne									
AT-5.1.1 Wyjaśnić heurystyki testowe.	K2				X		X		
AT-5.1.2 Podać przykłady mnemoników testowych związanych ze zwinnym wytwarzaniem oprogramowania.	K2				X		X		
AT-5.1.3 Wyjaśnić wędrówki testowe.	K2				X		X		

Cele nauczania – Poziom Zaawansowany Tester Zwinny	Poziom K	B01	B02	B03	B04	B05	B06	B07	B08
AT-5.1.4 Przeanalizować historyki użytkownika i opowieści w celu utworzenia kart opisu testu.	K4								X
AT-5.1.5 Zastosować testowanie eksploracyjne w celu wsparcia testowania w zwinnym wytwarzaniu oprogramowania.	K3				X		X		
5.2 Testowanie wspomagane									
AT-5.2.1 Wyjaśnić testowanie grupowe.	K2				X		X		
AT-5.2.2 Wyjaśnić testowanie w parach.	K2				X		X		
AT-5.2.3 Wyjaśnić testowanie intuicyjne.	K2				X		X		
5.3 Zapachy testów									
AT-5.3.1 Użyć zapachów testów do oceny jakości przypadków testowych.	K3				X		X		
6. Automatyzacja testów i narzędzia testowe (30 min.)									
6.1 Automatyzacja testów w zwinnym wytwarzaniu oprogramowania									
AT-6.1.1 Rozróżniać podejścia do automatyzacji testów stosowane w zwinnym wytwarzaniu oprogramowania.	K2							X	
6.2 Narzędzia testowe w zwinnym wytwarzaniu oprogramowania									
AT-6.2.1 Podać przykłady narzędzi testowych pomocnych w testowaniu zwinnym.	K2							X	

13. Dodatek E – nota wydania

Jest to główna wersja wydania sylabusa, dlatego nie podano szczegółowych zmian w stosunku do poprzedniej wersji. Zamiast tego, ponieważ różnica między wersją 1.0 a wersją 2.0 jest znacząca, poniżej opisano ogólny charakter i powód zmian.

Ponieważ Agile jest obecnie standardową metodą wytwarzania oprogramowania, podstawy metodyk zwinnych nie są już opisane w celach nauczania. Zamiast tego podstawy te zostały omówione we wstępie do rozdziału 1.

Część treści z sylabusa Tester Zwinny w. 1.0 została przeniesiona do sylabusa Poziomu Podstawowego w. 4.0. W związku z tym treści te nie są uwzględnione w aktualnym sylabusie Tester Zwinny w. 2.0. Dotyczy to podejścia najpierw test, piramidy testów, kwadrantów testowych, niezależności testowania, tworzenia historyjek użytkownika opartego na współpracy, retrospektyw oraz szacowania nakładu pracy związanego z testowaniem. W niniejszym sylabusie niektóre z tych treści zostały rozszerzone (np. kwadranty testowe na poziomie K4).

Wprowadzono nowe tematy, takie jak mnemoniki, heurystyki, wędrówki testowe, stroniczości, metryki pokrycia do raportowania, testowanie holistyczne, testerzy jednorazowi, testowanie grupowe, testowanie intuicyjne, *storyboarding*, *testboarding* i mapowanie przykładów, a niektóre treści omówione w sylabusie Poziomu Podstawowego v4.0 zostały rozszerzone (np. testowanie eksploracyjne i podejście „cały zespół”).

14. Dodatek F – znaki towarowe

ISTQB® jest zarejestrowanym znakiem towarowym International Software Testing Qualifications Board. TMMi® jest zarejestrowanym znakiem towarowym Fundacji TMMi. TPI-Next® jest zarejestrowanym znakiem towarowym Sogeti, Niderlandy.

15. Indeks

5W1H, 46
arkusz sesji testowej, 47
automatyzacja testów, 55
błąd zgodności, 39
Cel-Pytanie-Metryka, 33
ciche wydanie, 33
cykl PDCA, 34
czas realizacji testów, 33
DevOps, 36
diagram pętli przyczynowo-skutkowych, 34
doskonalenie procesu testowego, 32
efekt potwierdzenia, 38
efekt zakotwiczenia, 38
eXtreme Programming (XP), 14
FEW HICCUPPS, 44, 48
generalizacja, 22
heurystyka testowa, 43
historyjka użytkownika, 38
I SLICED UP FUN, 44
inżynieria chaosu, 33
inżynieria wymagań, 40
Kanban, 14
karta opisu testu, 45, 47
kryteria akceptacji, 36, 40, 46
kwadranty testowe, 27
lista kontrolna, 37
Manifest Agile, 13
mapa myśli, 37
mapowanie przykładów, 37
mapowanie strumienia wartości, 34
metryka, 32
mnemonik testowy, 44
monitorowanie testów, 30
motywowanie interesariuszy, 23
nadzór nad testami, 31
narzędzia testowe, 56
niestabilny test, 33
obserwowalność, 29
piramida testów, 55
planowanie iteracji, 28
planowanie testów, 28
planowanie wydania, 28
podejście, 34
podejście cały zespół, 22
podział historyjki użytkownika, 39
pokrycie, 31
potok CI/CD, 30
projektowa strategia testowa, 29
przetątnik funkcji, 33
przesunięcie w lewo, 36, 40
raportowanie testów, 31
RCRCRC, 44
retrospektywa, 33
ryzyko, 50
Scrum, 14
sesja testowania eksploracyjnego, 47
SFDIPOT, 44
specjalizacja, 22
specyfikacja przez przykłady, 36
społeczność praktyków, 33
spotkanie na stojąco, 31
storyboarding, 37
stronniczość, 38
TERMS, 44
test kompleksowy, 16
testboarding, 37
tester jednorazowy, 25
testowanie w parach, 34
testowanie eksploracyjne, 44, 47
testowanie formalne, 17
testowanie grupowe, 48
testowanie holistyczne, 18
testowanie intuicyjne, 49
testowanie regresji, 19
testowanie w parach, 49
testy regresji, 36
typ testu, 16
ustrukturyzowana rozmowa, 37
warsztaty ryzyka, 34
wędrowka testowa, 44
wsparcie programistów, 24
wydanie kanarkowe, 33
wyrocznia testowa, 48
wytwarzanie sterowane testami
 akceptacyjnymi, 36
wytwarzanie sterowane zachowaniem, 36
zapach testu, 50
żyjące wymagania, 36